Penetration Testing Report

Full Name: EMMANUEL ATEJI
Program: HCPT
Date: 22TH FEBRURARY 2025

Introduction

This report document hereby describes the proceedings and results of a Black Box security assessment conducted against the **Week 2 Labs**. The report hereby lists the findings and corresponding best practice mitigation actions and recommendations.

1. Objective

The objective of the assessment was to uncover vulnerabilities in the **Week 2 Labs** and provide a final security assessment report comprising vulnerabilities, remediation strategy and recommendation guidelines to help mitigate the identified vulnerabilities and risks during the activity.

2. Scope

This section defines the scope and boundaries of the project.

Application Name	Lab 1 - Insecure Direct Object References Lab 2 - SQL Injection

3. Summary

Outlined is a Black Box Application Security assessment for the **Week {2} Labs**.

Total number of Sub-labs: 16 Sub-labs

High	Medium	Low
{4}	{7}	{5}

High - **4 Sub-labs with high difficulty level**

Medium - **7 Sub-labs with Medium difficulty level**

Low - **5 Sub-labs with low difficulty level**

1. Insecure Direct Object References

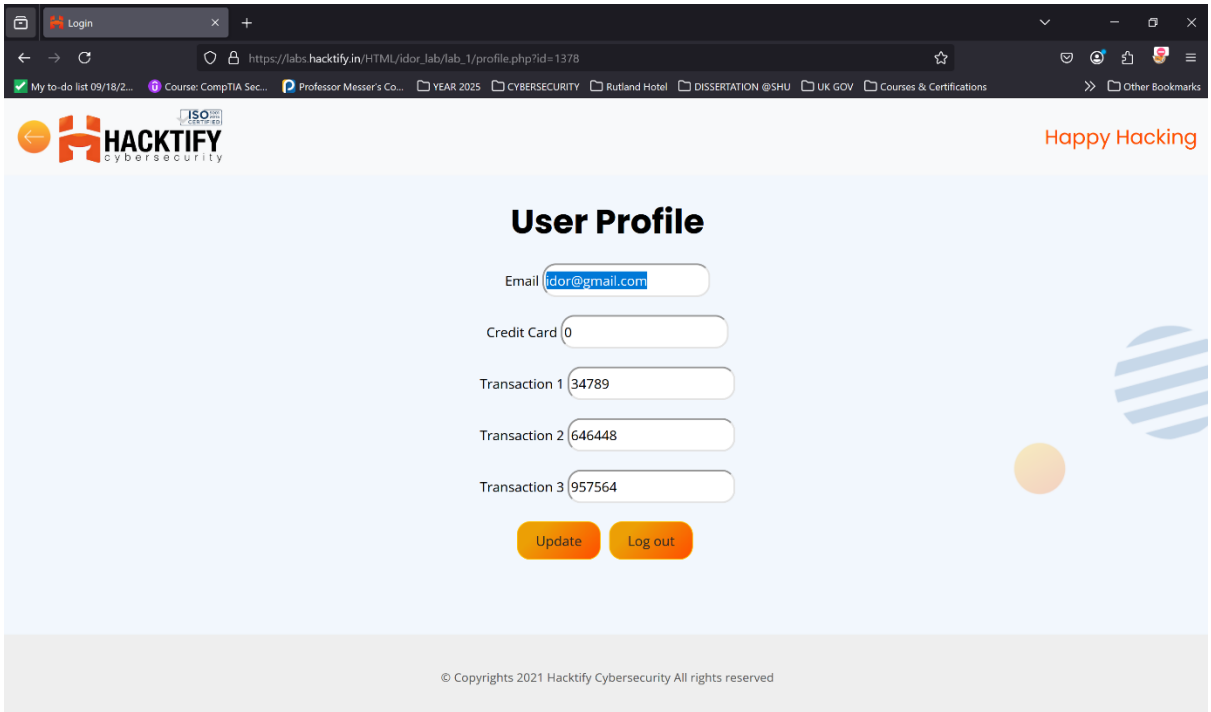
1.1. Give me my amount!!

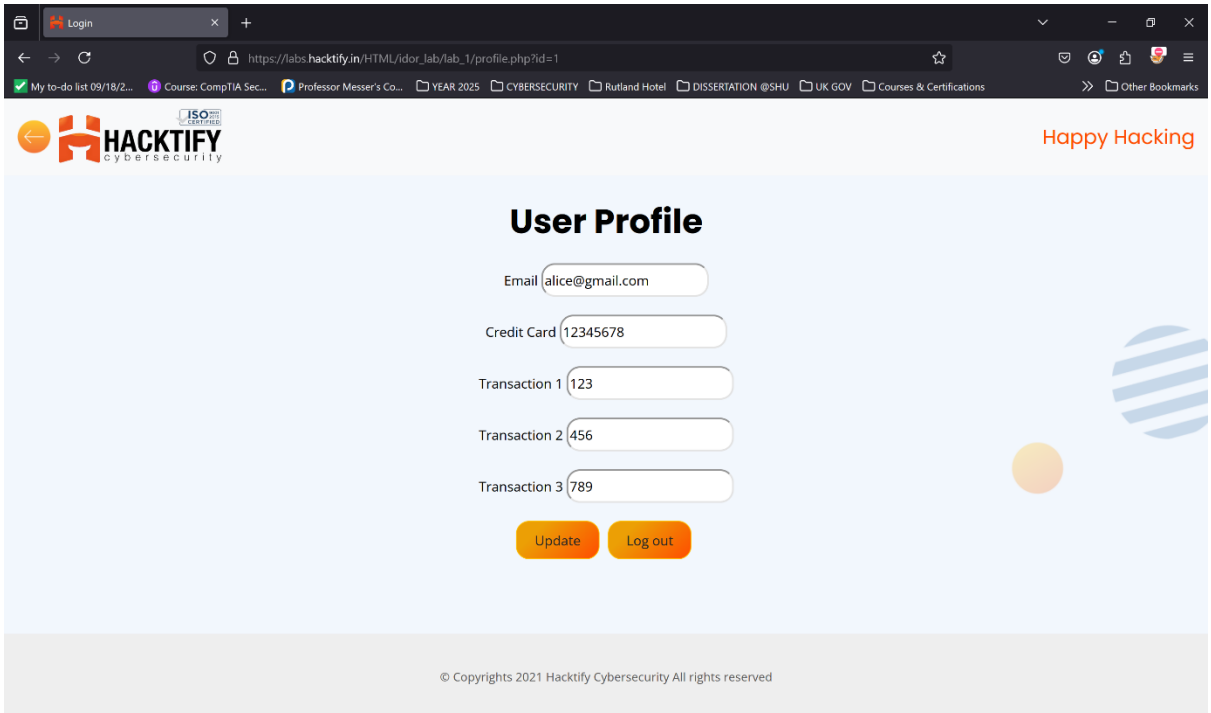
Reference	Risk Rating
Sub-lab-1: Give me my amount!!	**Low**
Tools Used	
Web Browser is used to find the vulnerability.	
Vulnerability Description	
Insecure direct object reference (IDOR) is an access control vulnerability where invalidated user input can be used for unauthorized access to resources or operations. IDOR vulnerability exists on the user profile page where changing the ID parameter in the URL (profile.php?id=X) allows unauthorized access to other users' sensitive information. In this case, by simply manipulating the ID value in the URL from 1378 to 1 or 7, an attacker can view other users' emails, credit card numbers, and transaction details without authentication. The application fails to verify whether the requesting user has proper authorization to access the requested profile data.	
How It Was Discovered	
Manual Analysis	
Vulnerable URLs	
https://labs.hacktify.in/HTML/idor_lab/lab_1/index.php	
Consequences of not Fixing the Issue	
1. **Unauthorized data access** - Attackers can view sensitive personal and financial information of all platform users by simply iterating through ID numbers 2. **Financial fraud** - Exposed credit card numbers and transaction IDs could enable financial fraud and unauthorized transactions 3. **Identity theft** - Compromised personal information could lead to identity theft and account takeovers 4. **Regulatory violations** - Non-compliance with data protection regulations (GDPR, PCI DSS) resulting in significant financial penalties and legal liability	
Suggested Countermeasures	
• Implement proper access control mechanisms that validate the user's session and authorization before allowing access to any profile data. • Use indirect reference maps or non-guessable Identifiers where the actual database identifiers are replaced with temporary tokens that have no meaning outside the application. • Apply the principle of least privilege and Role-based Access Control (RBAC) by ensuring users can only access their own profile information, requiring proper authentication for any data access.	
References	
https://portswigger.net/web-security/access-control/idor	

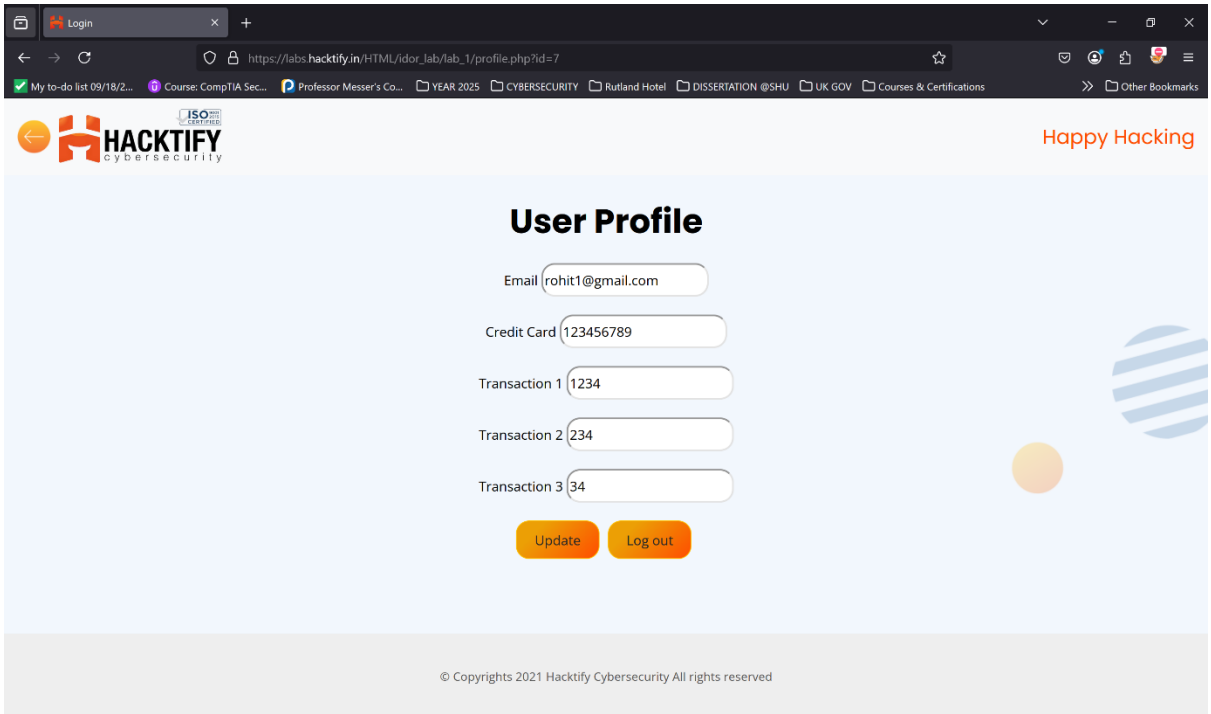
https://www.imperva.com/learn/application-security/insecure-direct-object-reference-idor/
https://cheatsheetseries.owasp.org/cheatsheets/Insecure_Direct_Object_Reference_Prevention_Cheat_Sheet.html
https://www.spiceworks.com/it-security/vulnerability-management/articles/insecure-direct-object-reference-idor/
https://blog.detectify.com/best-practices/owasp-top-10-insecure-direct-object-reference-4/

Proof of Concept

This section contains the proof of the above vulnerabilities as the screenshot of the vulnerability of the lab




Login
https://labs.hacktify.in/HTML/idor_lab/lab_1/profile.php?id=1
My to-do list 09/18/2...
Course: CompTIA Sec...
Professor Messer's Co...
YEAR 2025
CYBERSECURITY
Rutland Hotel
DISSERTATION @SHU
UK GOV
Courses & Certifications
Other Bookmarks
HACKTIFY
cybersecurity
Happy Hacking
User Profile
Email alice@gmail.com
Credit Card 12345678
Transaction 1 123
Transaction 2 456
Transaction 3 789
Update
Log out
© Copyrights 2021 Hacktify Cybersecurity All rights reserved


Login
https://labs.hacktify.in/HTML/idor_lab/lab_1/profile.php?id=7
My to-do list 09/18/2...
Course: CompTIA Sec...
Professor Messer's Co...
YEAR 2025
CYBERSECURITY
Rutland Hotel
DISSERTATION @SHU
UK GOV
Courses & Certifications
Other Bookmarks
HACKTIFY
cybersecurity
Happy Hacking
User Profile
Email rohit1@gmail.com
Credit Card 123456789
Transaction 1 1234
Transaction 2 234
Transaction 3 34
Update
Log out
© Copyrights 2021 Hacktify Cybersecurity All rights reserved

1.2. Stop polluting my params!

Reference	Risk Rating
Sub-lab-2: Stop polluting my params!	**Medium**

Tools Used

Browser "Inspector" is used to find the vulnerability.

Vulnerability Description

Insecure direct object reference (IDOR) is an access control vulnerability where invalidated user input can be used for unauthorized access to resources or operations. IDOR vulnerability exists in the user profile page of the web application.
In this case, it reveal that changing the ID parameter in the URL (profile.php?id=5, profile.php?id=939, profile.php?id=1&id=2) allows unauthorized access and modifications to different user profiles. This happens because the application uses client-supplied input to directly reference objects without proper authorization checks. The application accepts the user-controlled parameter (id) and retrieves the corresponding user profile data without verifying if the requesting user has permission to view that information.

How It Was Discovered

Manual Analysis

Vulnerable URLs

https://labs.hacktify.in/HTML/idor_lab/lab_2/index.php

Consequences of not Fixing the Issue

1. **Unauthorized data access** - Attackers can view sensitive personal information of other users (email addresses, names).
2. **Account takeover** - Attackers might leverage exposed information to reset passwords or gain unauthorized access.
3. **Mass data harvesting** - Automated scripts could enumerate all IDs to extract complete user databases.
4. **Privacy violations and regulatory non-compliance** - Unauthorized access to PII could lead to violations of privacy regulations like GDPR.

Suggested Countermeasures

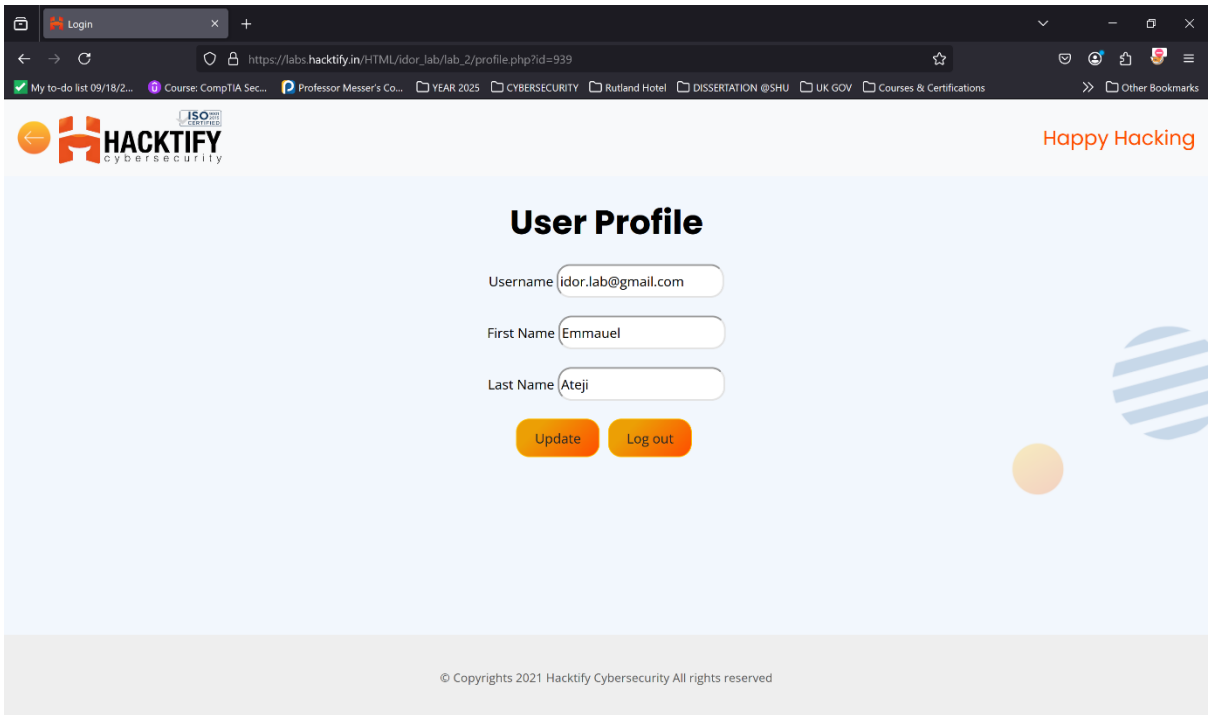
1. **Implement proper authorization checks** - Verify the user's session corresponds to the requested profile ID or enforce role-based access controls.
2. **Use indirect reference maps** - Replace direct object references with temporary, session-specific tokens that map to actual database identifiers.
3. **Implement server-side request validation** - Add input validation and sanitization for all URL parameters, rejecting any attempt to manipulate IDs.

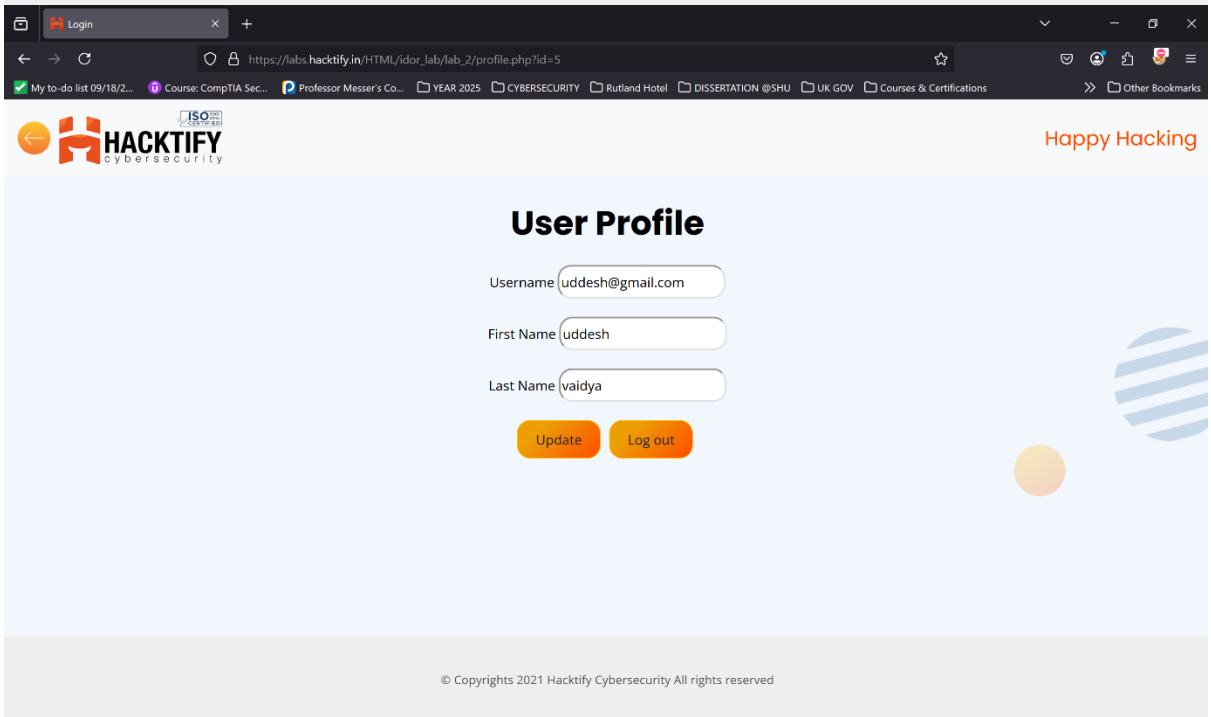
References

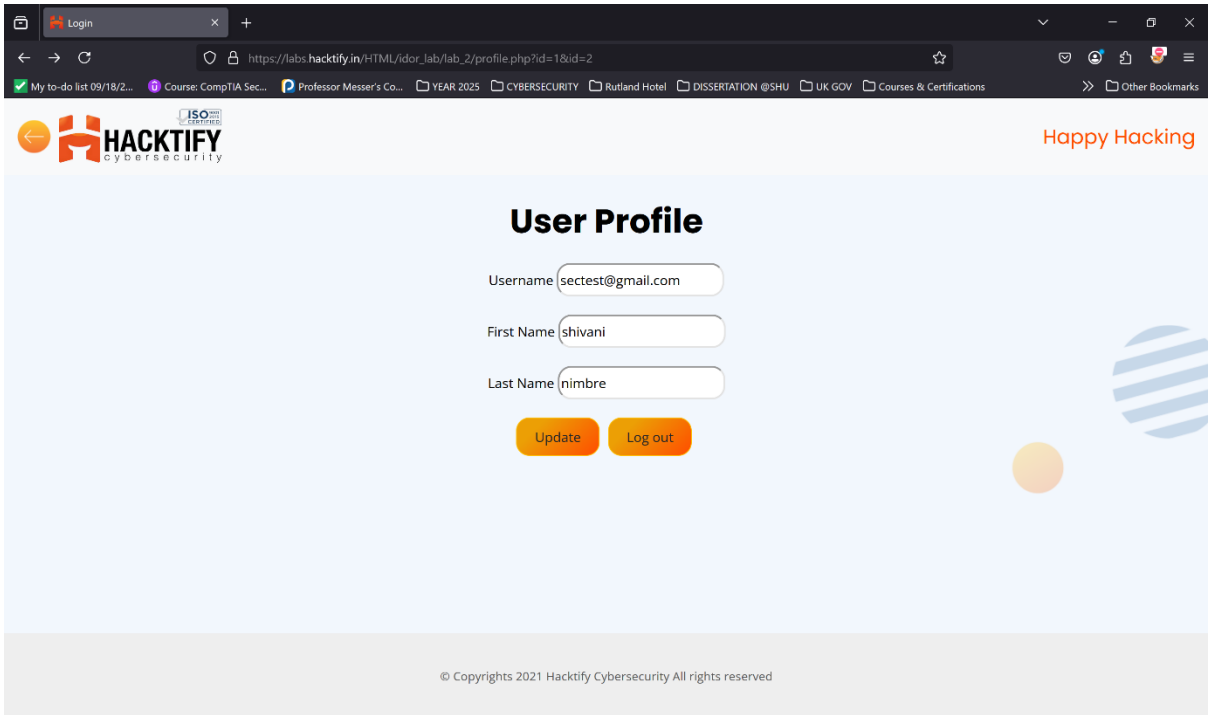
https://www.invicti.com/learn/insecure-direct-object-references-idor/
https://medium.com/@rakesh.testengr/owasp-security-shepherd-indirect-object-reference-challenges-write-up-9d608ba59e36
https://portswigger.net/web-security/access-control/idor
https://en.wikipedia.org/wiki/Insecure_direct_object_reference

Proof of Concept

This section contains the proof of the above vulnerabilities as the screenshot of the vulnerability of the lab







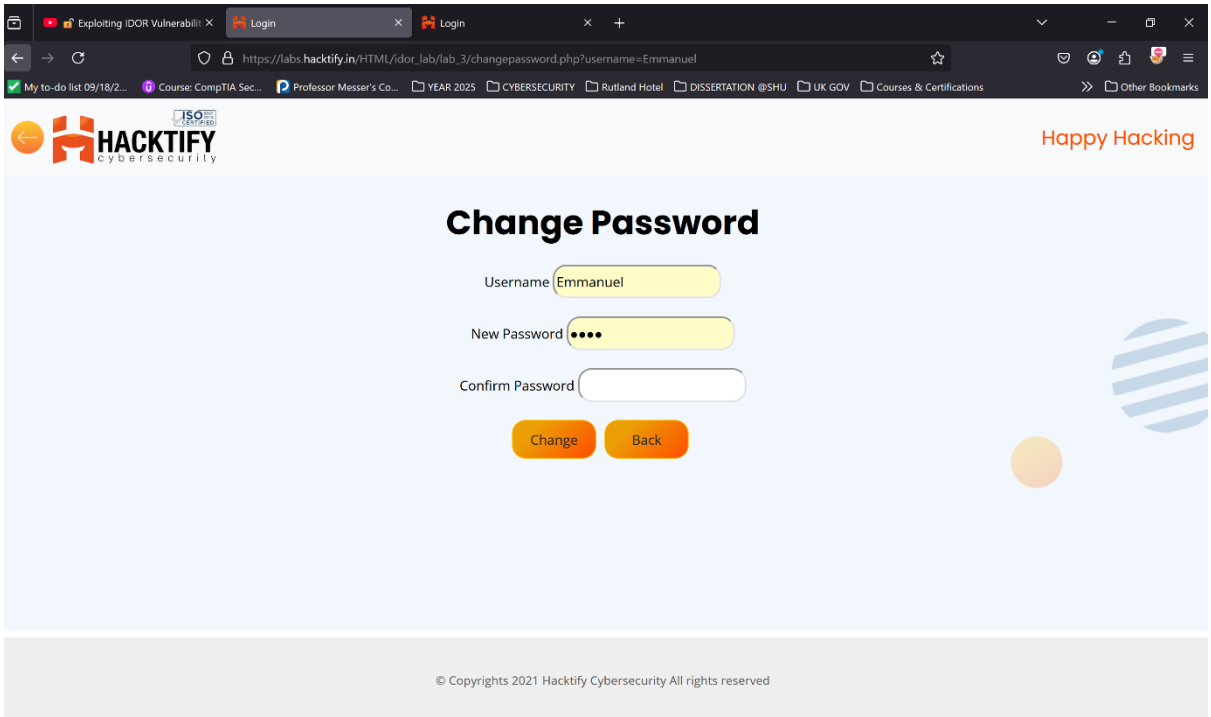
1.3. Someone changed my Password!

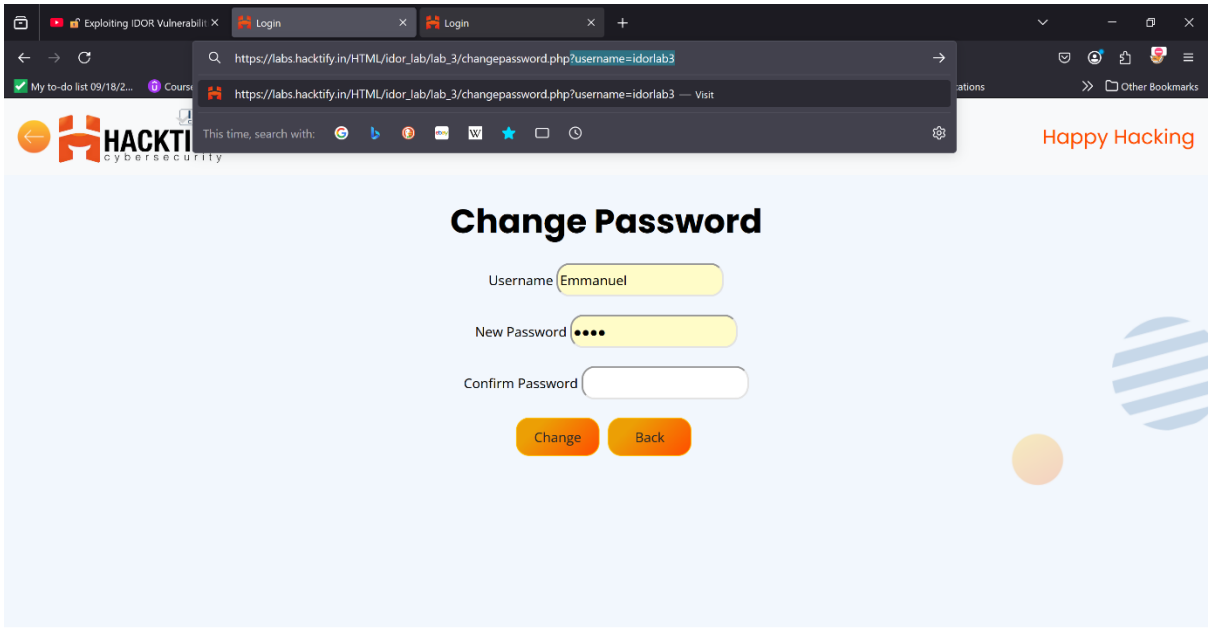
Reference	Risk Rating
Sub-lab-3: Someone changed my Password!	**Medium**
Tools Used	
Web Browser is used to find the vulnerability.	
Vulnerability Description	
This is an Insecure Direct Object Reference (IDOR) vulnerability. The application uses the username parameter in the URL (https://labs.hackify.in/HTML/idor_lab/lab_3/changepassword.php?username=idorlab3) to identify the user whose password needs to be changed. The vulnerability exists because the application doesn't properly validate or authorize whether the currently logged-in user (Emmanuel) has the right to change the password for the user specified in the username parameter (in this case, "idorlab3"). As a result, an attacker (Emmanuel) can change the username parameter to another user's username and reset their password without proper authorization.	
How It Was Discovered	
Manual Analysis	
Vulnerable URLs	
https://labs.hacktify.in/HTML/idor_lab/lab_3/lab_3.php	
Consequences of not Fixing the Issue	
1. **Unauthorized Account Access:** Attackers can take over user accounts by changing passwords without permission, leading to data breaches and identity theft. 2. **Data Breaches:** Attackers could gain access to sensitive personal or financial information stored in the compromised accounts. 3. **Reputational Damage:** If the vulnerability is exploited, it can lead to a loss of trust from users and damage the organization's reputation.	

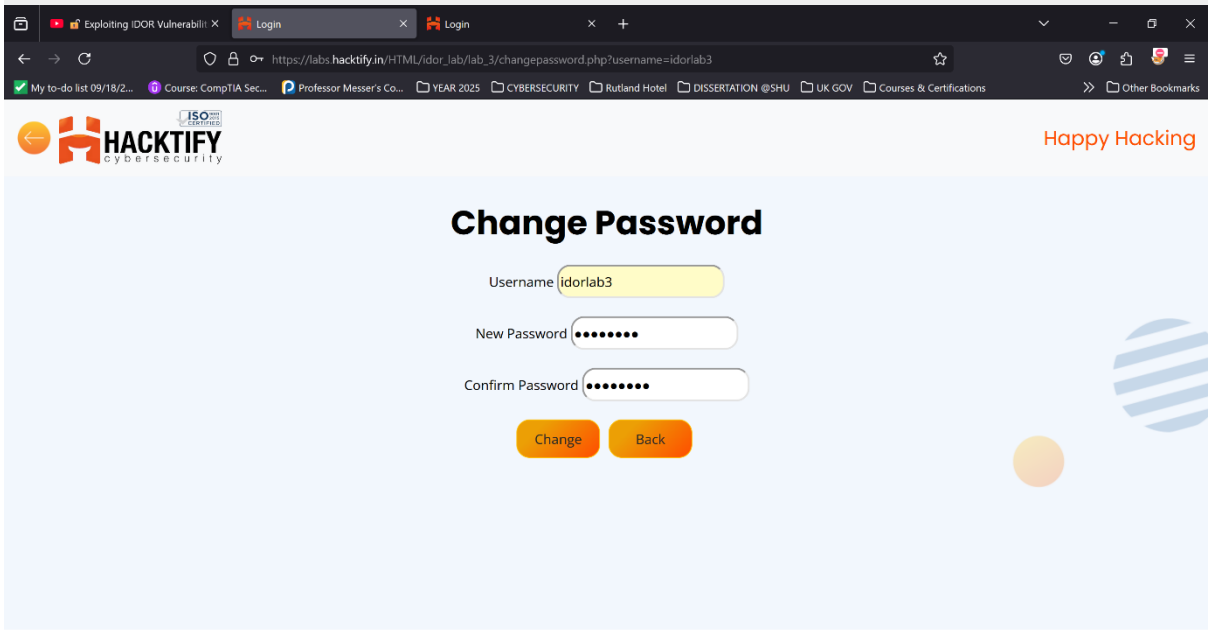
4. **Privilege Escalation:** In some cases, attackers might be able to escalate their privileges by targeting administrator or other high-privilege accounts.
Suggested Countermeasures
1. **Implement Proper Authorization Checks:** Before allowing a password change, the application should verify that the currently logged-in user has the authority to reset the password for the specified user. This might involve checking if the user is an administrator or if they are the owner of the account being modified. 2. **Use unpredictable session-specific tokens:** Implement a session-specific, unpredictable token (like a CSRF token) when initiating the password reset request. This can help ensure that the user is authorized to perform the action. 3. **Implement Access Control Lists (ACLs):** Define granular permissions for different user roles, specifying what data each role can access and modify
References
1. OWASP IDOR: https://owasp.org/www-project-top-ten/ 2. PortSwigger IDOR: https://portswigger.net/web-security/idor 3. Acunetix IDOR: https://www.acunetix.com/blog/web-security/insecure-direct-object-references-idor/

Proof of Concept

This section contains the proof of the above vulnerabilities as the screenshot of the vulnerability of the lab




Exploiting IDOR Vulnerabili
Login
Login
https://labs.hacktify.in/HTML/idor_lab/lab_3/changepassword.php?username=idorlab3
https://labs.hacktify.in/HTML/idor_lab/lab_3/changepassword.php?username=idorlab3 — Visit
This time, search with:
Other Bookmarks
HACKTI
cybersecurity
Happy Hacking
Change Password
Username Emmanuel
New Password ••••
Confirm Password
Change
Back
© Copyrights 2021 Hacktify Cybersecurity All rights reserved

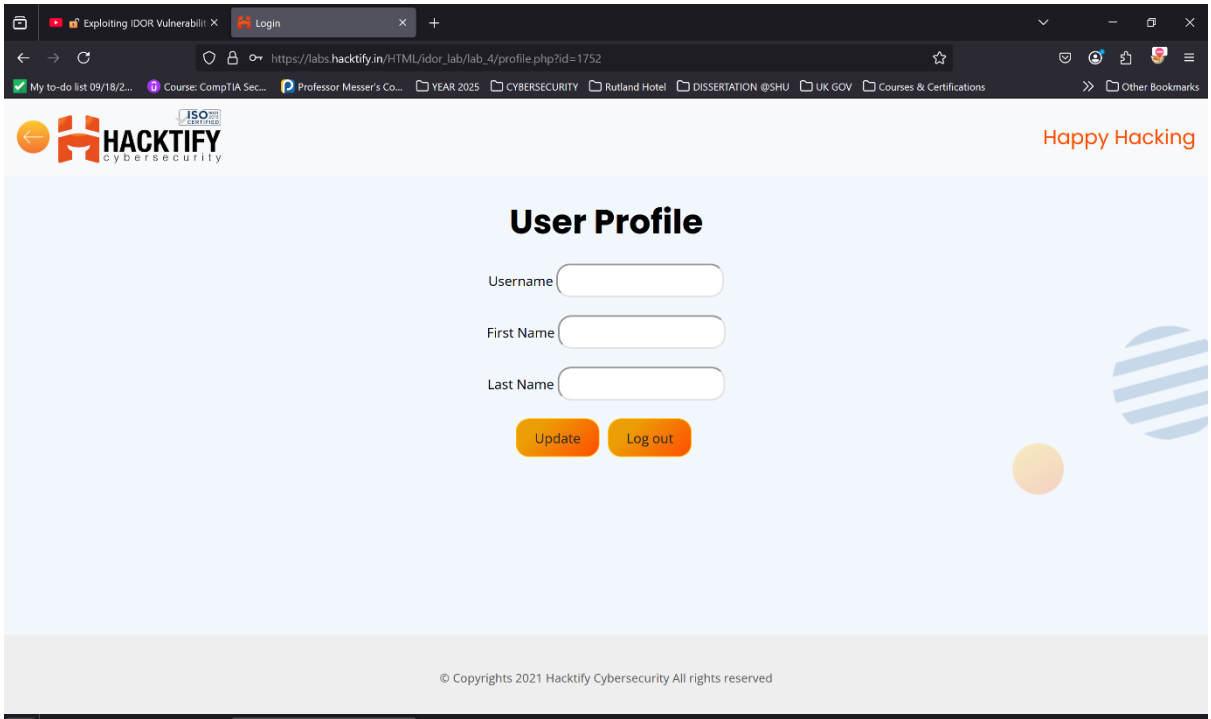

Exploiting IDOR Vulnerabili
Login
Login
https://labs.hacktify.in/HTML/idor_lab/lab_3/changepassword.php?username=idorlab3
My to-do list 09/18/2...
Course: CompTIA Sec...
Professor Messer's Co...
YEAR 2025
CYBERSECURITY
Rutland Hotel
DISSERTATION @SHU
UK GOV
Courses & Certifications
Other Bookmarks
HACKTIFY
cybersecurity
Happy Hacking
Change Password
Username idorlab3
New Password ••••••••
Confirm Password ••••••••
Change
Back
© Copyrights 2021 Hacktify Cybersecurity All rights reserved

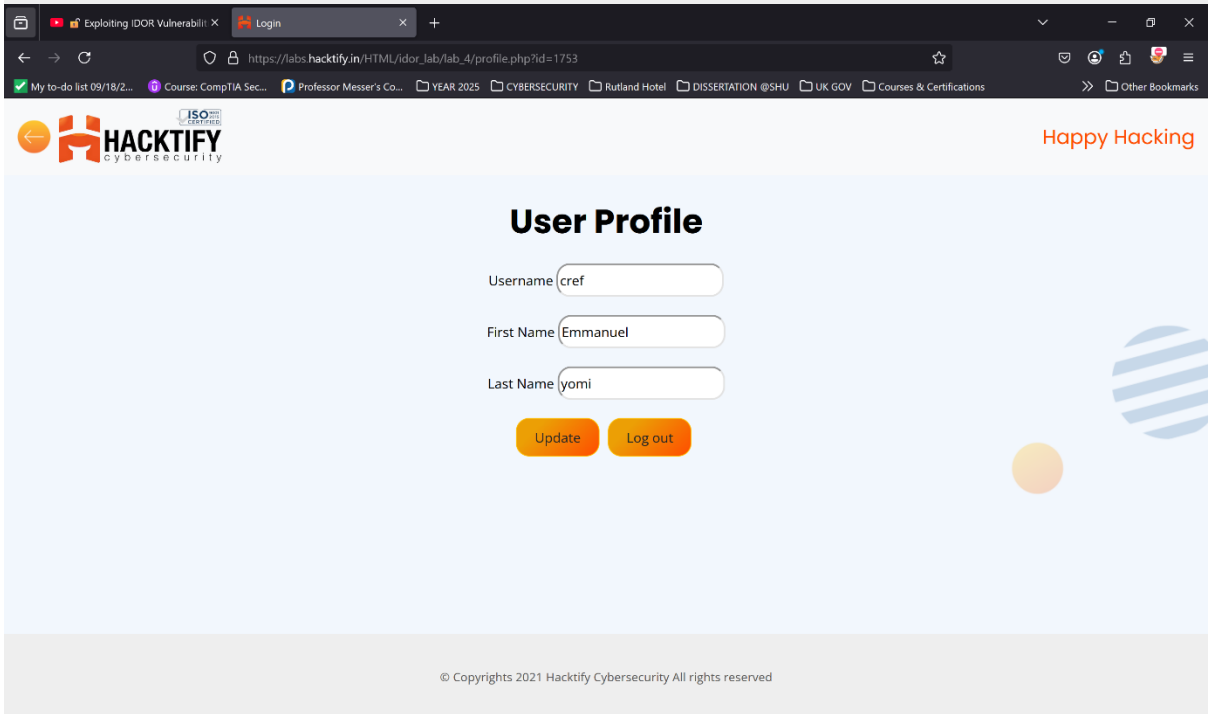
1.4. Change your methods!

Reference	Risk Rating
Sub-lab-4: Change your methods!	**Medium**
Tools Used	
Web Browser is used to find this vulnerability	
Vulnerability Description	
Insecure Direct Object Reference (IDOR): The application uses the id parameter in the URL to directly access user profiles. It fails to verify whether the currently logged-in user has the authority to access the profile associated with the provided id. This allows attackers to potentially view and modify the data of other users by simply changing the id value in the URL. In this case, from ?id=1752 to ?id=1753	
How It Was Discovered	
Manual Analysis	
Vulnerable URLs	
https://labs.hacktify.in/HTML/idor_lab/lab_4/lab_4.php	
Consequences of not Fixing the Issue	
1. **Data Breach:** Attackers can access sensitive user information (usernames, first names, last names, potentially more). 2. **Account Takeover:** If other profile details (email, etc.) are exposed and modifiable, attackers could potentially use the exposed information to perform a password reset and take over accounts. 3. **Data Manipulation:** Attackers can modify user profiles, leading to data corruption, defacement, or other malicious activities. 4. **Reputational Damage:** Discovery of such a vulnerability can damage the reputation of the application/company and erode user trust.	
Suggested Countermeasures	
1. **Implement Authorization Checks:** Before displaying or allowing modification of user profiles, verify that the logged-in user has the right to access and modify the data associated with the requested id. This could involve checking if the id matches the user's own ID, or if the user has administrative privileges. 2. **Use Indirect Object References:** Instead of using the direct database ID in the URL, use a non-guessable, session-specific token or GUID. Map this token on the server-side to the actual user ID after proper authorization. 3. **Implement Access Control Lists (ACLs):** Define granular permissions for different user roles, specifying what data each role can access and modify.	
References	
1. OWASP IDOR: https://owasp.org/www-project-top-ten/OWASP_Top_Ten_2021/ (Look for the Broken Access Control section) 2. PortSwigger IDOR: https://portswigger.net/web-security/access-control/idor 3. Acunetix IDOR: https://www.acunetix.com/blog/web-security-vulnerabilities/what-is-idor/ 4. SANS Institute on Access Control:https://www.sans.org/information-security/glossary/access-control	

Proof of Concept

This section contains the proof of the above vulnerabilities as the screenshot of the vulnerability of the lab





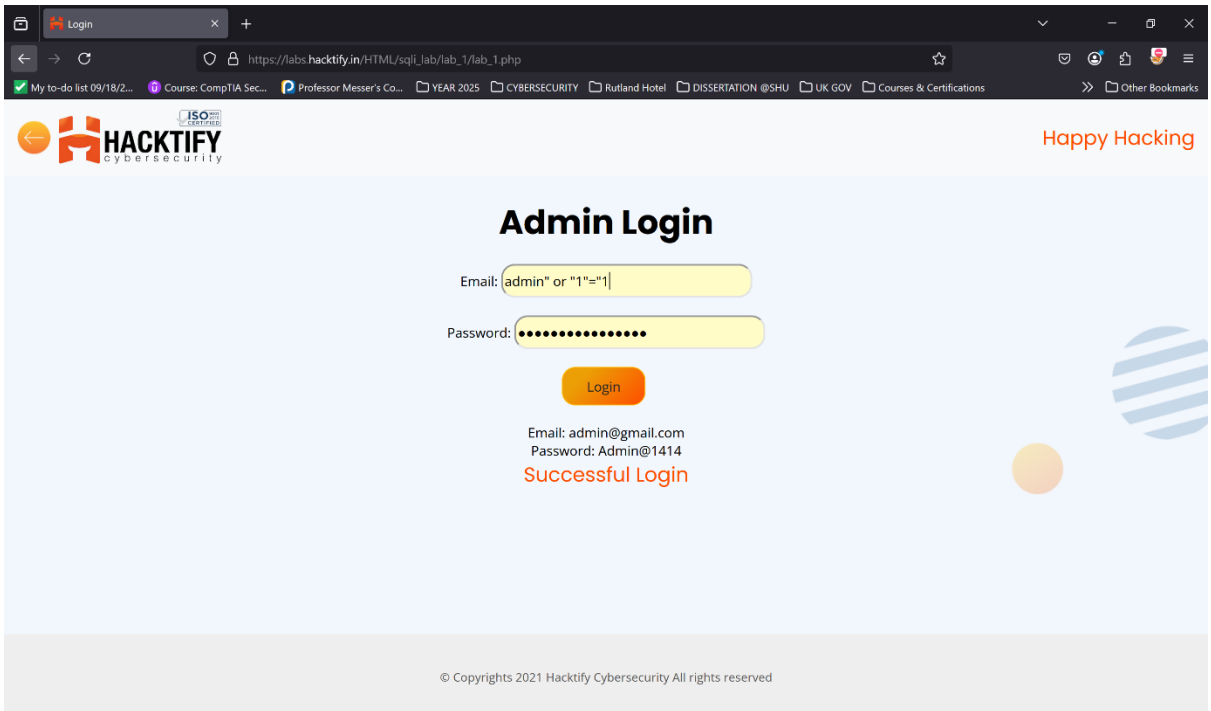
2. SQL Injection Labs

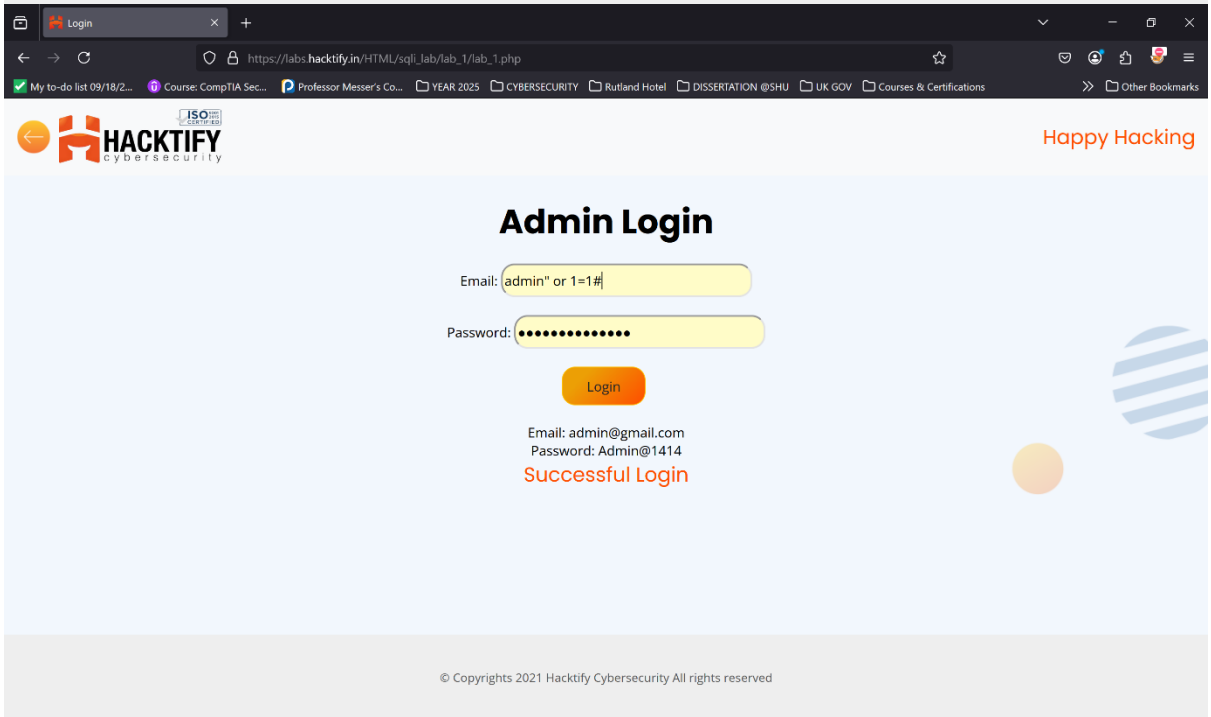
2.1. Strings & Errors Part 1!

Reference	Risk Rating
Sub-lab-1: Strings & Errors Part 1!	**Low**
Tools Used	
Web Browser, GitHub is used to find this vulnerability	
Vulnerability Description	
This is a classic SQL injection vulnerability in the login form. The application fails to properly sanitize user input before incorporating it into SQL queries. The payload admin" or "1"="1 in the username field creates a condition that always evaluates to true, bypassing authentication. This works because the application likely constructs an SQL query similar to SELECT * FROM users WHERE username="[user input]" AND password="[password input]". When the malicious input is inserted, the resulting query becomes logically flawed, allowing authentication bypass without knowing valid credentials.	
How It Was Discovered	
Manual Analysis	
Vulnerable URLs	
https://labs.hacktify.in/HTML/sqli_lab/lab_1/lab_1.php	
Consequences of not Fixing the Issue	
1. **Unauthorized Access**: Attackers can bypass authentication and gain admin access without legitimate credentials 2. **Data Breach**: Full access to sensitive user data and backend database content. 3. **System Compromise**: Potential to execute advanced SQL commands that could modify or delete database content. 4. **Credential Theft:** Ability to extract password hashes that could be cracked offline.	
Suggested Countermeasures	
1. **Implement Prepared Statements/Parameterized Queries:** Replace direct string concatenation with parameterized queries to separate SQL code from user input. 2. **Input Validation:** Apply strict input validation to reject special characters and enforce input format rules. 3. **Implement Proper Error Handling:** Return generic error messages instead of revealing SQL syntax errors that help attackers refine their injection attempts	
References	
https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html https://owasp.org/www-community/attacks/SQL_Injection https://portswigger.net/web-security/sql-injection https://cve.mitre.org/cgi-bin/cvekey.cgi?keyword=SQL+injection https://github.com/swisskyrepo/PayloadsAllTheThings/blob/master/SQL%20Injection/Intruder/Auth_Bypass.txt	

Proof of Concept

This section contains the proof of the above vulnerabilities as the screenshot of the vulnerability of the lab



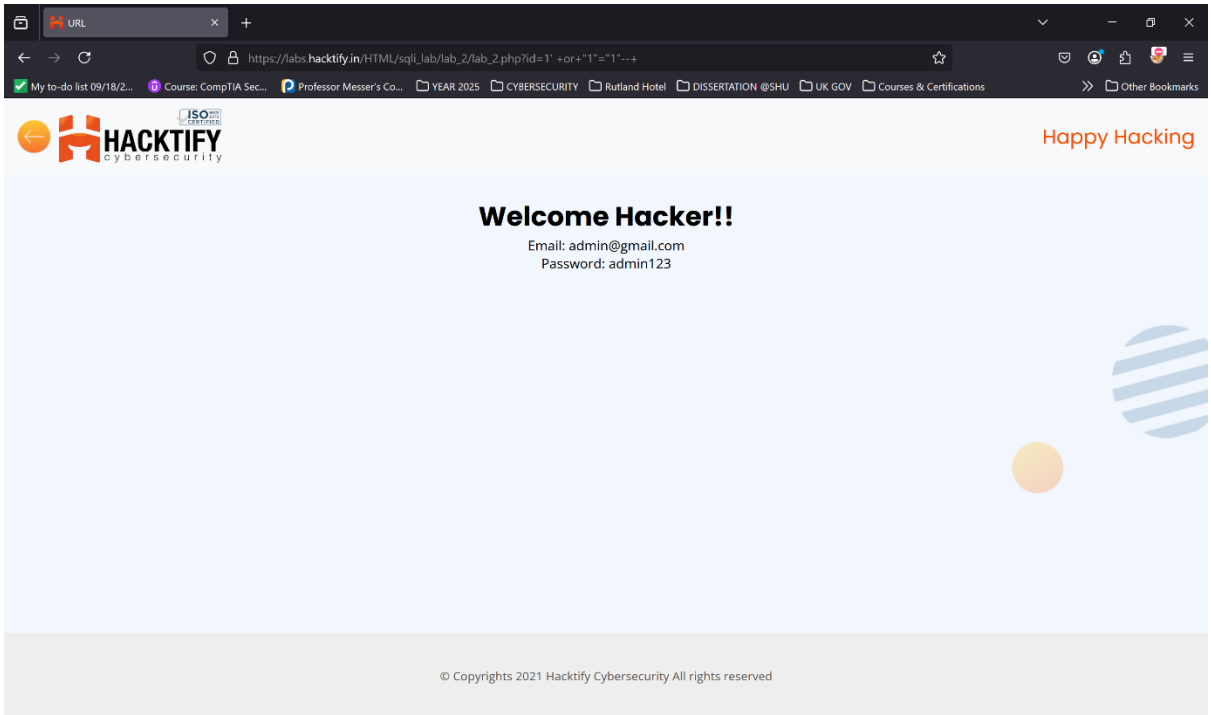


2.2. Strings & Errors Part 2!

Reference	Risk Rating
Sub-lab-2: Strings & Errors Part 2!	**Low**
Tools Used	
Web Browser was used for this vulnerability.	
Vulnerability Description	
This is a classic SQL injection vulnerability where the application fails to properly sanitize user input from the URL parameter. The payload ?id=1' +or+"1"="1"--+ exploits this by injecting SQL code that creates a condition that always evaluates to true, bypassing authentication. The vulnerability exists because the application directly concatenates user input into SQL queries without proper validation or parameterization. This particular injection uses string manipulation with single quotes and the OR operator to ensure the query returns results.	
How It Was Discovered	
Manual Analysis	
Vulnerable URLs	
https://labs.hacktify.in/HTML/sqli_lab/lab_2/lab_2.php	
Consequences of not Fixing the Issue	
1. **Unauthorized access to sensitive data -** Attackers can bypass authentication to view restricted information. 2. **Data breach** - Complete database extraction possible through techniques like UNION-based injections. 3. **Data manipulation/corruption** - Attackers can potentially modify or delete database records 4. **Complete system compromise** - In some cases, SQL injection can lead to server-level access through file system operations.	
Suggested Countermeasures	
• Implement prepared statements/parameterized queries instead of direct string concatenation • Apply input validation and sanitization for all user-supplied data • Implement the principle of least privilege for database users in the application	
References	
https://owasp.org/www-community/attacks/SQL_Injection https://portswigger.net/web-security/sql-injection https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html https://www.hacksplaining.com/exercises/sql-injection	

Proof of Concept

This section contains the proof of the above vulnerabilities as the screenshot of the vulnerability of the lab



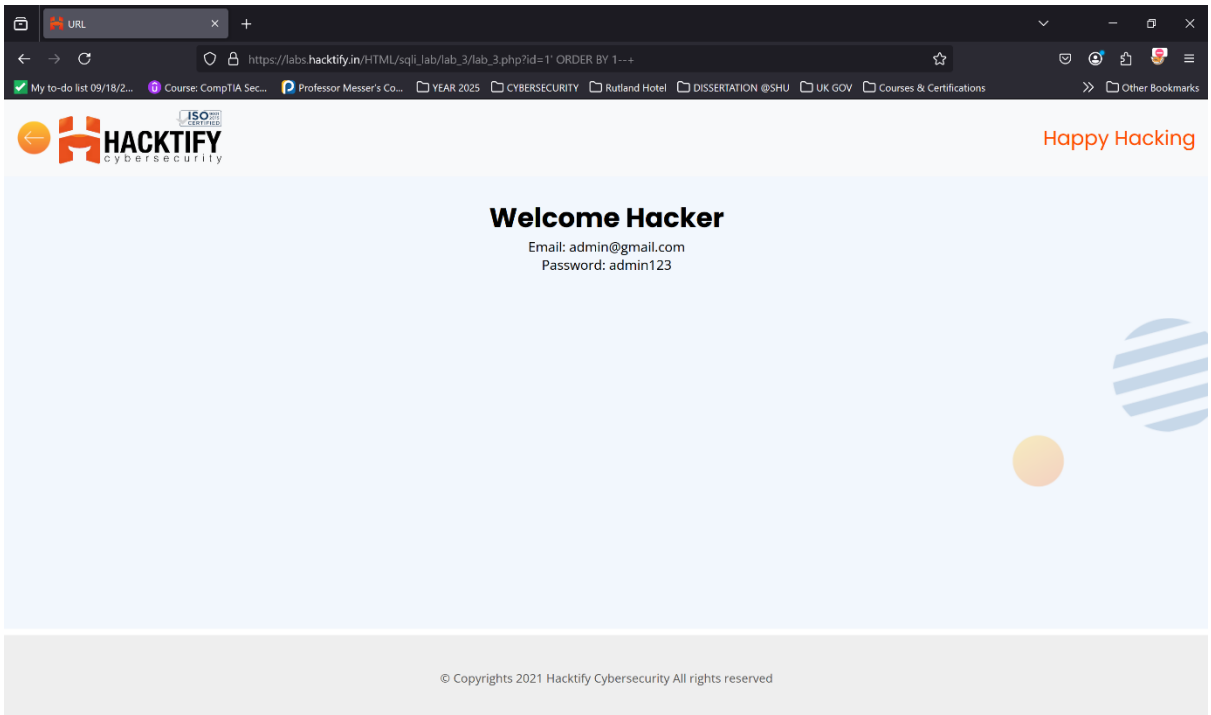
2.3. Strings & Errors Part 3!

Reference	Risk Rating
Sub-lab-3: Strings & Errors Part 3!	**Easy**
Tools Used	
Web Browser is used to find this vulnerability	
Vulnerability Description	
This is an error-based SQL Injection vulnerability where the application fails to properly sanitize user input in the "id" parameter. When a single quote is added to the parameter value (?id=1'), it breaks the SQL syntax, causing the database to return error messages. The ORDER BY payload is testing column enumeration, which helps determine the structure of the database query. The presence of this vulnerability suggests the application directly incorporates user input into SQL queries without proper parameterization. Payload: ?id=1' ORDER BY 1--+ ?id=1'	
How It Was Discovered	
Manual Analysis	
Vulnerable URLs	
https://labs.hacktify.in/HTML/sqli_lab/lab_3/lab_3.php	
Consequences of not Fixing the Issue	
1. **Unauthorized data access** - Attackers can extract sensitive information from the database 2. **Database manipulation** - Attackers can modify, delete, or insert data 3. **Authentication bypass** - Attackers might bypass login mechanisms 4. **Server compromise** - In some cases, this could lead to server-level access through techniques like file reading or command execution	

Suggested Countermeasures
1. Implement prepared statements/parameterized queries to separate SQL code from data 2. Apply input validation to reject suspicious characters 3. Use an ORM (Object-Relational Mapping) framework that handles SQL escaping automatically
References
https://owasp.org/www-community/attacks/SQL_Injection https://portswigger.net/web-security/sql-injection https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html https://www.acunetix.com/websitesecurity/sql-injection/

Proof of Concept

This section contains the proof of the above vulnerabilities as the screenshot of the vulnerability of the lab

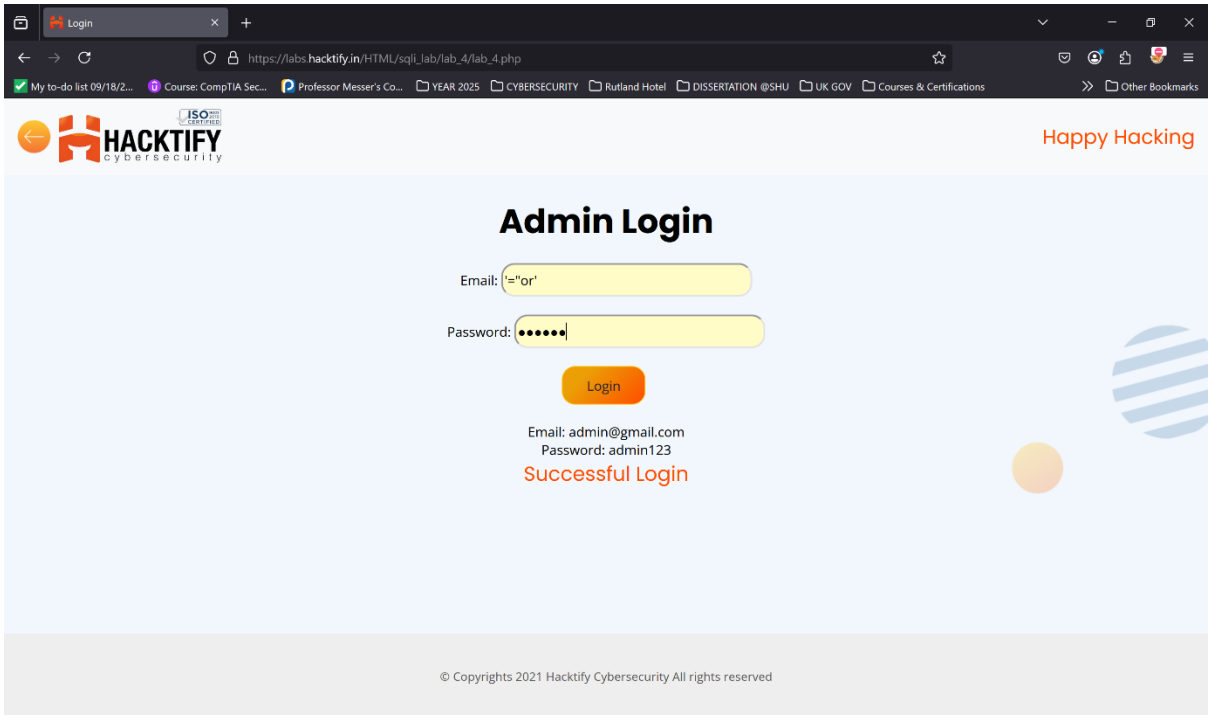


2.4. Let's Trick 'em!

Reference	Risk Rating
Sub-lab-4: Let's Trick 'em!	**Medium**
Tools Used	
Web Browser is used to find this vulnerability	
Vulnerability Description	
This is a classic SQL injection vulnerability in the login form. The application appears to be constructing an SQL query without properly sanitizing or parameterizing user input. The payload '="or' works because it manipulates the SQL query's WHERE clause to create a condition that always evaluates to true. When the application processes this input, it likely creates a query like: SELECT * FROM users WHERE email=''="or'' AND password=''="or'' The odd syntax with quotes and equals signs creates a condition that always returns true, allowing authentication bypass without valid credentials.	
How It Was Discovered	
Manual Analysis	
Vulnerable URLs	
https://labs.hacktify.in/HTML/sqli_lab/lab_4/lab_4.php	
Consequences of not Fixing the Issue	
1. **Unauthorized Administrative Access** - Attackers can bypass authentication and gain admin privileges 2. **Data Breach** - Unauthorized access to sensitive user information and system data 3. **Complete Database Compromise** - The injection vulnerability could allow extraction of all database content 4. **System Takeover** - Depending on database privileges, attackers might execute system commands	
Suggested Countermeasures	
• **Implement Prepared Statements/Parameterized Queries** - Use parameterized queries instead of string concatenation to build SQL statements • **Input Validation** - Implement strict input validation for all user-controllable data • **Implement Proper Authentication Mechanisms** - Replace vulnerable authentication logic with secure frameworks	
References	
https://owasp.org/www-community/attacks/SQL_Injection https://portswigger.net/web-security/sql-injection https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html https://www.hacksplaining.com/exercises/sql-injection	

Proof of Concept

This section contains the proof of the above vulnerabilities as the screenshot of the vulnerability of the lab



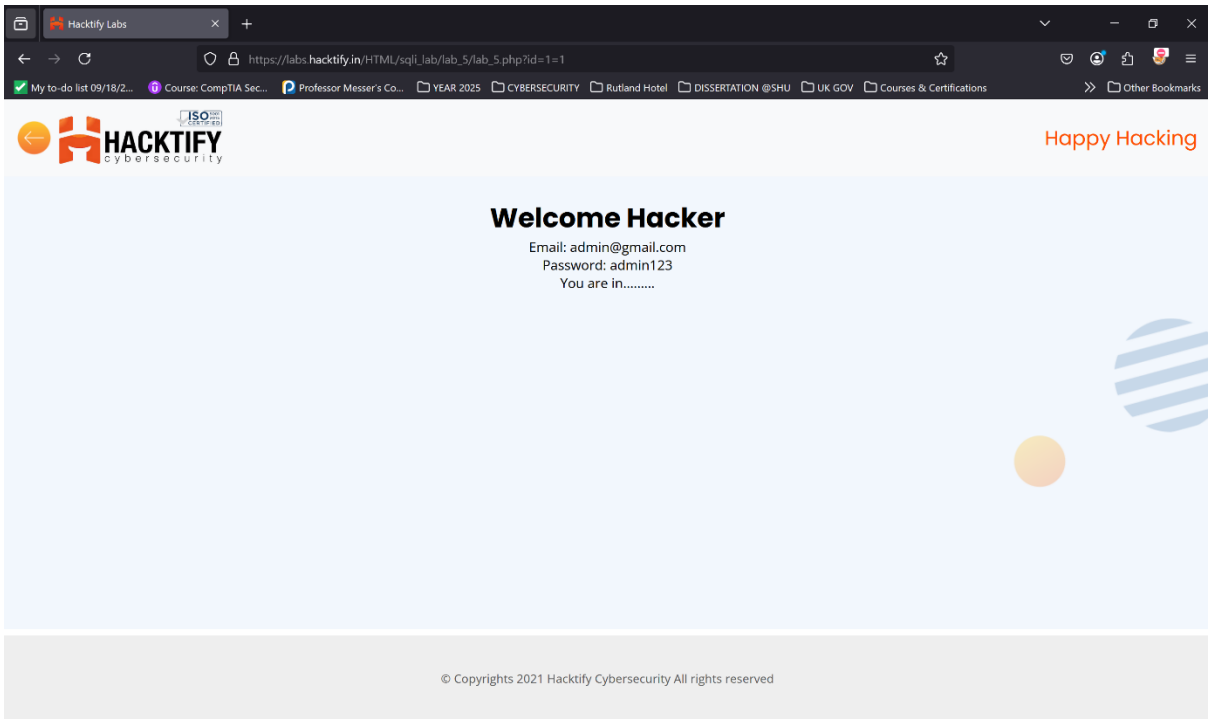
2.5. Booleans and Blind!

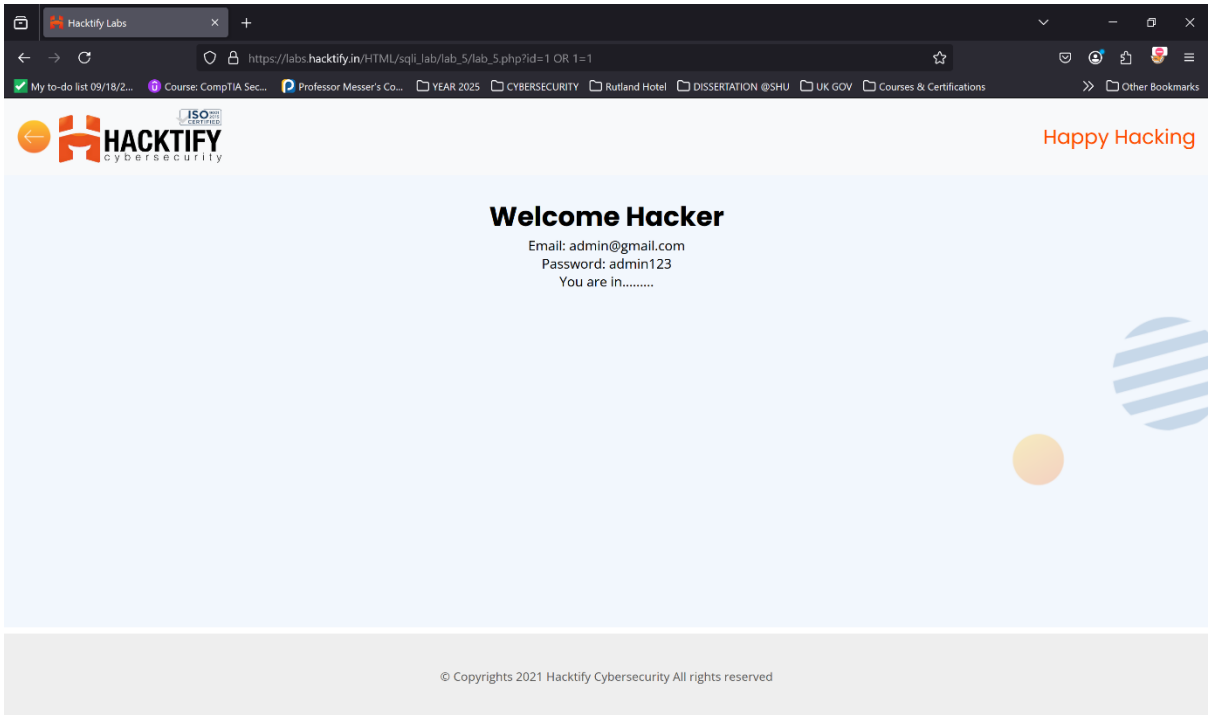
Reference	Risk Rating
Sub-lab-5: Booleans and Blind!	**High**
Tools Used	
Web Browser is used to find the vulnerability.	
Vulnerability Description	
The application is vulnerable to Boolean-based blind SQL injection in the id parameter. In this case: • The application accepts direct SQL syntax in URL parameters without proper sanitization • When changing from ?id=1=1 to ?id=1 OR 1=1, the page content remains identical, indicating that the SQL query executes successfully with injected logic • The application appears to directly insert user input into SQL queries without parameterization • The vulnerability allows an attacker to modify the logic of SQL queries by injecting Boolean conditions	
How It Was Discovered	
Manual Analysis	
Vulnerable URLs	
https://labs.hacktify.in/HTML/sqli_lab/lab_5/lab_5.php	
Consequences of not Fixing the Issue	
• **Unauthorized data access** - Attackers can extract sensitive information from the database • **Data breach** - Complete database exfiltration is possible through blind techniques • **Authentication bypass** - The OR 1=1 technique can bypass login controls • **Database corruption/destruction** - Attackers could potentially execute destructive SQL commands	
Suggested Countermeasures	
• Implement prepared statements/parameterized queries to separate SQL logic from data • Apply input validation and strong type checking for all user-supplied parameters • Implement the principle of least privilege for database connections to limit potential damage	

References
https://owasp.org/www-community/attacks/SQL_Injection https://portswigger.net/web-security/sql-injection/blind https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html https://cwe.mitre.org/data/definitions/89.html

Proof of Concept

This section contains the proof of the above vulnerabilities as the screenshot of the vulnerability of the lab





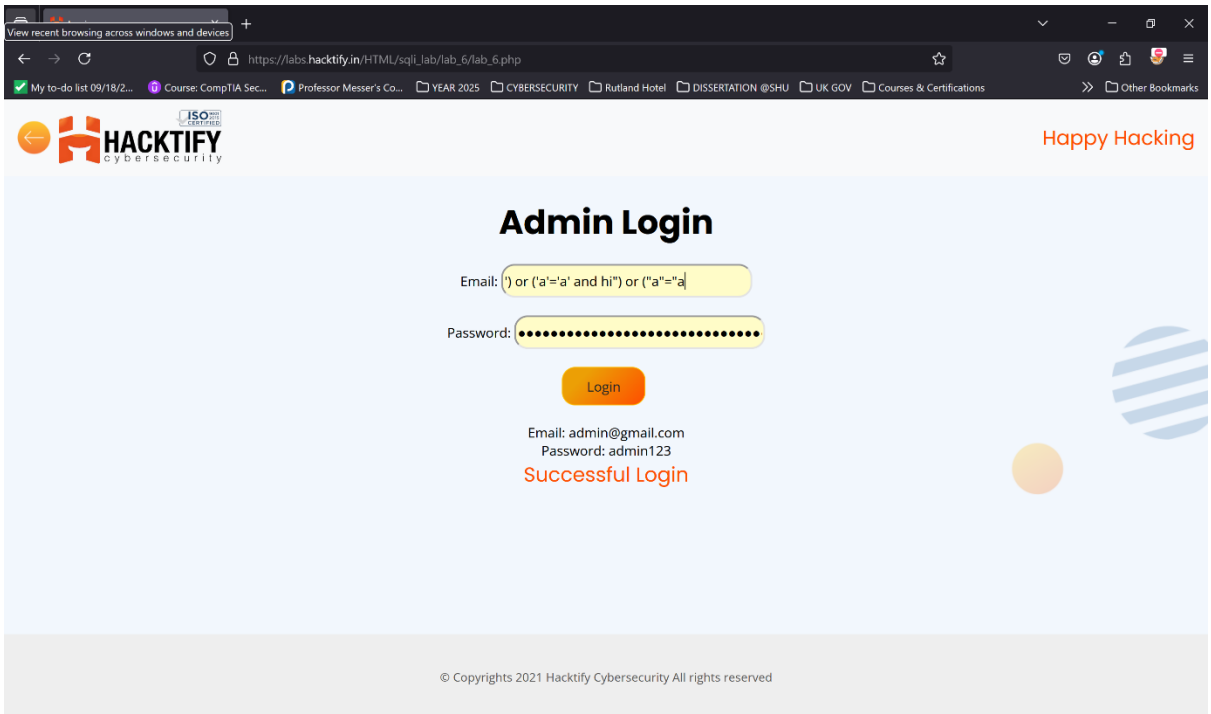
2.6. Error Based : Tricked

Reference	Risk Rating
Sub-lab-6: Error Based : Tricked	**Medium**
Tools Used	
Web Browser is used to find the vulnerability.	
Vulnerability Description	
This is an error-based SQL injection vulnerability where the application fails to properly sanitize user input in the login form. The payload ') or ('a'='a' and hi") or ("a"="a exploits boolean logic to create a tautology (always true condition). This tricks the application into executing the SQL query with an injected condition that always evaluates to true, bypassing authentication. The shows successful login bypass where credentials admin@gmail.com and admin123 were revealed without legitimate authentication.	
How It Was Discovered	
Manual Analysis	
Vulnerable URLs	
https://labs.hacktify.in/HTML/sqli_lab/lab_6/lab_6.php	
Consequences of not Fixing the Issue	
1. Unauthorized access to administrative interfaces and sensitive data. 2. Complete authentication bypass allowing attackers to gain admin privileges 3. Potential for data theft including user credentials and personal information 4. Database manipulation/corruption through further SQL commands	
Suggested Countermeasures	
1. Implement prepared statements with parameterized queries to separate SQL code from data 2. Apply input validation and sanitization for all user-provided data before using it in database queries 3. Use an Object-Relational Mapping (ORM) library that automatically handles SQL escaping	
References	

https://owasp.org/www-community/attacks/SQL_Injection
https://portswigger.net/web-security/sql-injection
https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html
https://cwe.mitre.org/data/definitions/89.html

Proof of Concept

This section contains the proof of the above vulnerabilities as the screenshot of the vulnerability of the lab



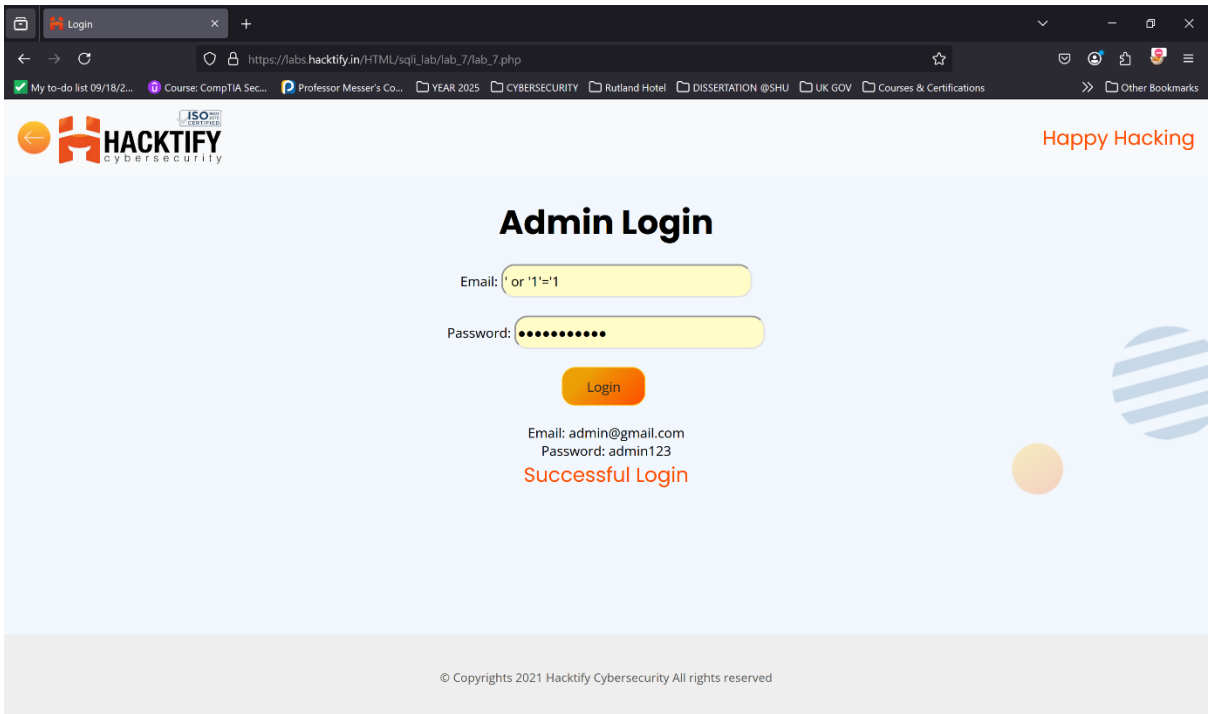
2.7. Errors and Post!

Reference	Risk Rating
Sub-lab-7: Errors and Post!	**Easy**
Tools Used	
Web Browser, Online URL Encoder is used to find the vulnerability.	
Vulnerability Description	
SQL Injection vulnerability in the login form. The application fails to properly sanitize user inputs in the email field, allowing attackers to inject SQL code. The payload ' or '1'='1 creates a condition that always evaluates to true, bypassing authentication logic. The application shows "Successful Login" despite not providing valid credentials, indicating the injection manipulated the query's WHERE clause to return all records.	
How It Was Discovered	
Manual Analysis	
Vulnerable URLs	

https://labs.hacktify.in/HTML/sqli_lab/lab_7/lab_7.php
Consequences of not Fixing the Issue
1. Unauthorized access to admin accounts and protected resources 2. Complete database compromise (data theft, modification, or deletion) 3. Potential for privilege escalation to higher system access 4. Regulatory compliance violations and potential legal consequences
Suggested Countermeasures
1. Implement prepared statements/parameterized queries instead of direct string concatenation 2. Apply input validation and sanitization using whitelist approach 3. Implement the principle of least privilege for database connections
References
https://owasp.org/www-community/attacks/SQL_Injection https://portswigger.net/web-security/sql-injection https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html https://cwe.mitre.org/data/definitions/89.html

Proof of Concept

This section contains the proof of the above vulnerabilities as the screenshot of the vulnerability of the lab

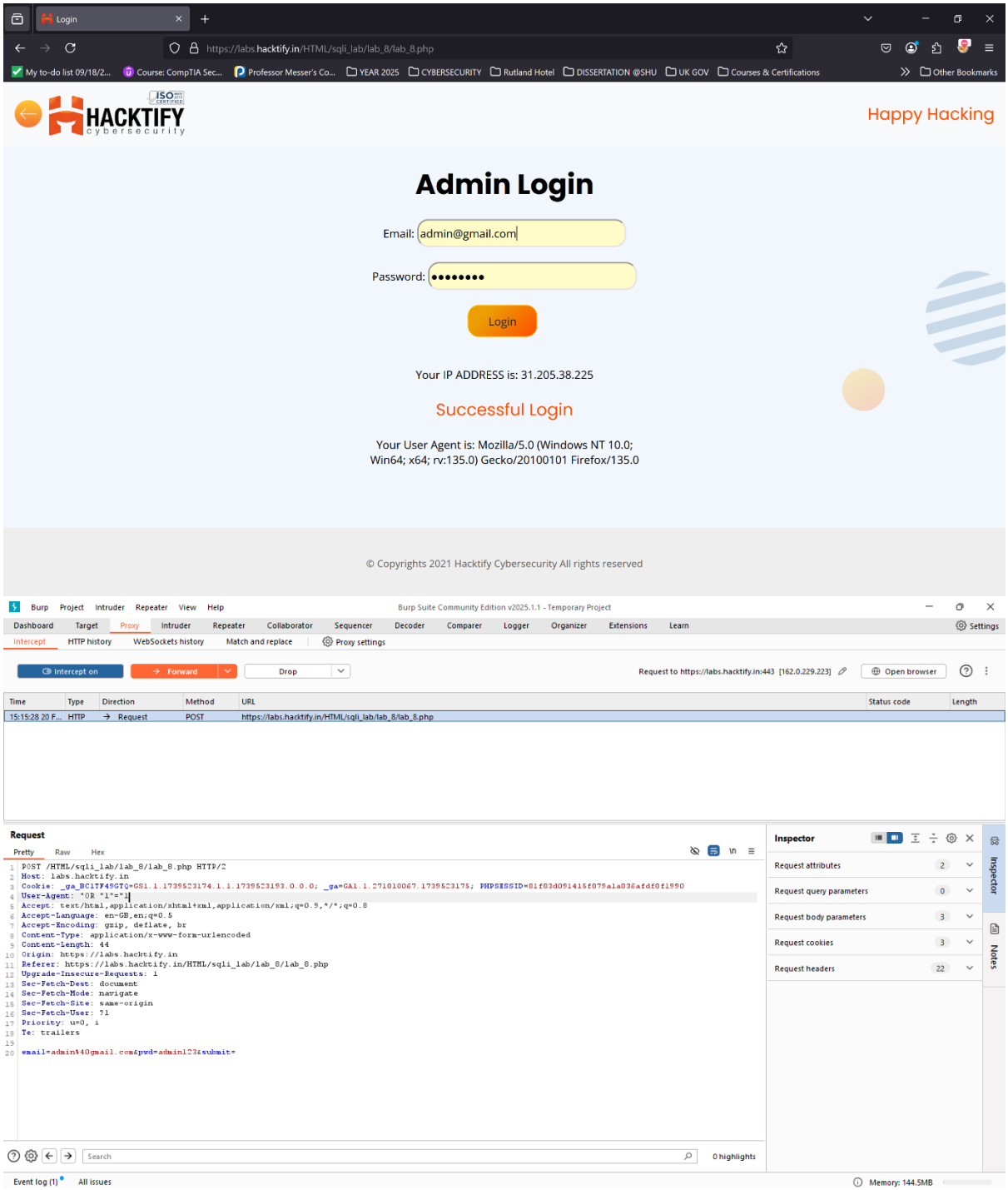


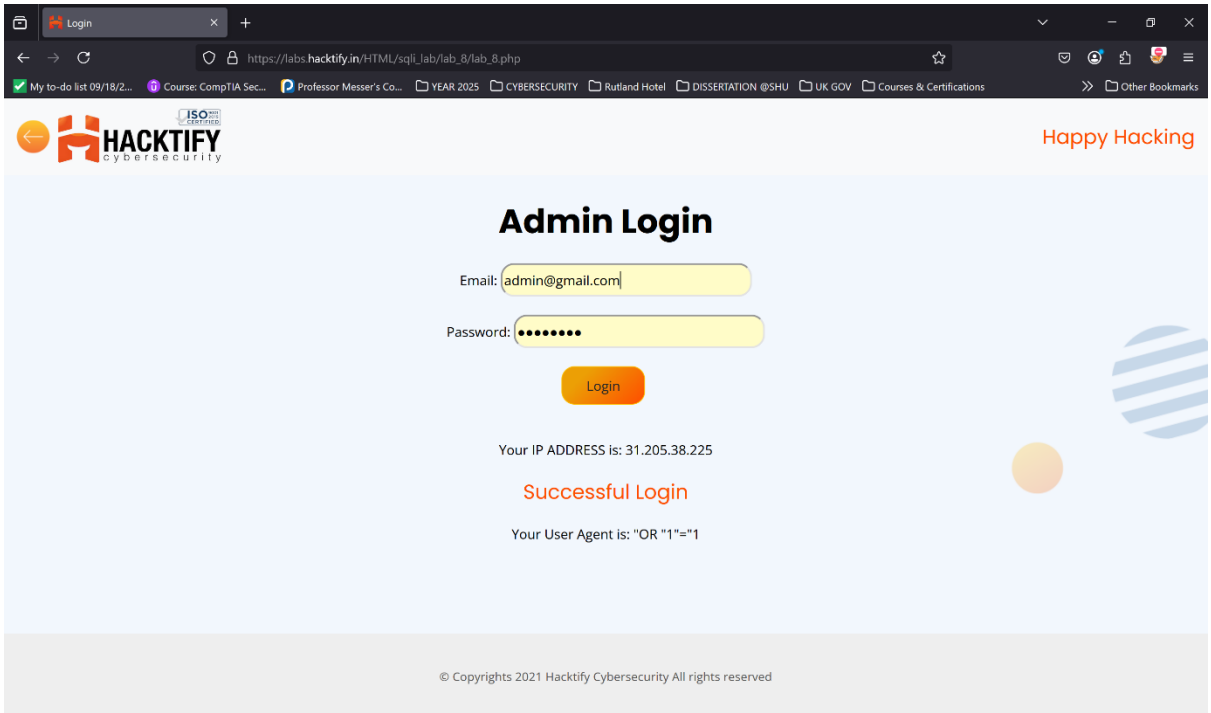
2.8. User Agents lead us!

Reference	Risk Rating
Sub-lab-8: User Agents lead us!	**High**
Tools Used	
Burp Suite Community Edition v2025.1.1, Web Browser are used to find this vulnerability.	
Vulnerability Description	
This is a SQL injection vulnerability in the User-Agent header processing. The application accepts input from the User-Agent header without proper sanitization and directly incorporates it into SQL queries. As shown, when the User-Agent was changed to "OR "1"="1, the authentication check was bypassed with a successful login message. This indicates that the User-Agent value is being inserted directly into a SQL query's WHERE clause, and the injected condition (OR 1=1) makes the query always return true.	
How It Was Discovered	
Manual Analysis	
Vulnerable URLs	
https://labs.hacktify.in/HTML/sqli_lab/lab_8/lab_8.php	
Consequences of not Fixing the Issue	
1. **Authentication Bypass**: Attackers can log in as any user including administrators without valid credentials 2. **Data Theft**: The vulnerability could be exploited to extract sensitive data from the database 3. **Data Manipulation**: Attackers could modify database content through SQL commands 4. **Database Compromise**: With more advanced payloads, attackers might gain access to the entire database server	
Suggested Countermeasures	
1. Implement parameterized queries or prepared statements to separate SQL code from data< 2. Apply input validation and sanitization on all HTTP headers, particularly User-Agent 3. Use the principle of least privilege for database connections to limit the damage if an injection occurs	
References	
https://owasp.org/www-community/attacks/SQL_Injection https://portswigger.net/web-security/sql-injection https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html https://cwe.mitre.org/data/definitions/89.html	

Proof of Concept

This section contains the proof of the above vulnerabilities as the screenshot of the vulnerability of the lab


Login
https://labs.hacktify.in/HTML/sqli_lab/lab_8/lab_8.php
HACKTIFY cybersecurity
Happy Hacking
Admin Login
Email: admin@gmail.com
Password: ••••••••
Login
Your IP ADDRESS is: 31.205.38.225
Successful Login
Your User Agent is: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:135.0) Gecko/20100101 Firefox/135.0
© Copyrights 2021 Hacktify Cybersecurity All rights reserved
Burp Suite Community Edition v2025.1.1 - Temporary Project
Intercept on
Forward
Drop
Request to https://labs.hacktify.in:443 [162.0.229.223]
Open browser
Time
Type
Direction
Method
URL
Status code
Length
15:15:28 20 F...
HTTP
Request
POST
https://labs.hacktify.in/HTML/sqli_lab/lab_8/lab_8.php
Request
Pretty
Raw
Hex
POST /HTML/sqli_lab/lab_8/lab_8.php HTTP/2
Host: labs.hacktify.in
Cookie: _ga_BC1TF45GTQ=GS1.1.1739523174.1.1.1739523193.0.0.0; _ga=GA1.1.271810067.1739523175; PHPSESSID=81f83d091415f879a1a936afdf9f1990
User-Agent: "OR "1"="1
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-GB,en;q=0.5
Accept-Encoding: gzip, deflate, br
Content-Type: application/x-www-form-urlencoded
Content-Length: 44
Origin: https://labs.hacktify.in
Referer: https://labs.hacktify.in/HTML/sqli_lab/lab_8/lab_8.php
Upgrade-Insecure-Requests: 1
Sec-Fetch-Dest: document
Sec-Fetch-Mode: navigate
Sec-Fetch-Site: same-origin
Sec-Fetch-User: ?1
Priority: u=0, i
Te: trailers
email=admin%40gmail.com&pwd=admin123&submit=
Inspector
Request attributes 2
Request query parameters 0
Request body parameters 3
Request cookies 3
Request headers 22
Search
0 highlights
Event log (1)
All issues
Memory: 144.5MB



2.9. Referer lead us!

Reference	Risk Rating
Sub-lab-9: Referer lead us!	**Medium**
Tools Used	
Burp Suite Community Edition v2025.1.1, Web Browser is used to find this vulnerability.	
Vulnerability Description	
This vulnerability exploits improper validation and sanitization of user-supplied data in the Referer header field of HTTP requests. The application appears to dynamically execute SQL queries based on the Referer field input. By injecting SQL commands such as 1" OR "1"="1, an attacker can bypass authentication and gain unauthorized access to the system. SQL injection vulnerabilities arise when user inputs are concatenated directly into SQL queries without proper escaping or parameterization. In this case, the injected payload modifies the authentication query, effectively bypassing any credential verification.	
How It Was Discovered	
Manual Analysis	
Vulnerable URLs	
https://labs.hacktify.in/HTML/sqli_lab/lab_9/lab_9.php	
Consequences of not Fixing the Issue	
1. **Authentication Bypass:** Attackers can log in as any user, including administrators, without needing valid credentials. 2. **Data Breach:** Sensitive user data, including passwords, financial records, and personal details, could be extracted from the database. 3. **Database Manipulation:** Attackers could delete, modify, or insert malicious data, potentially corrupting the entire system. 4. **Full System Compromise:** If the database privileges are high, attackers could execute OS-level commands, leading to complete server takeover.	
Suggested Countermeasures	

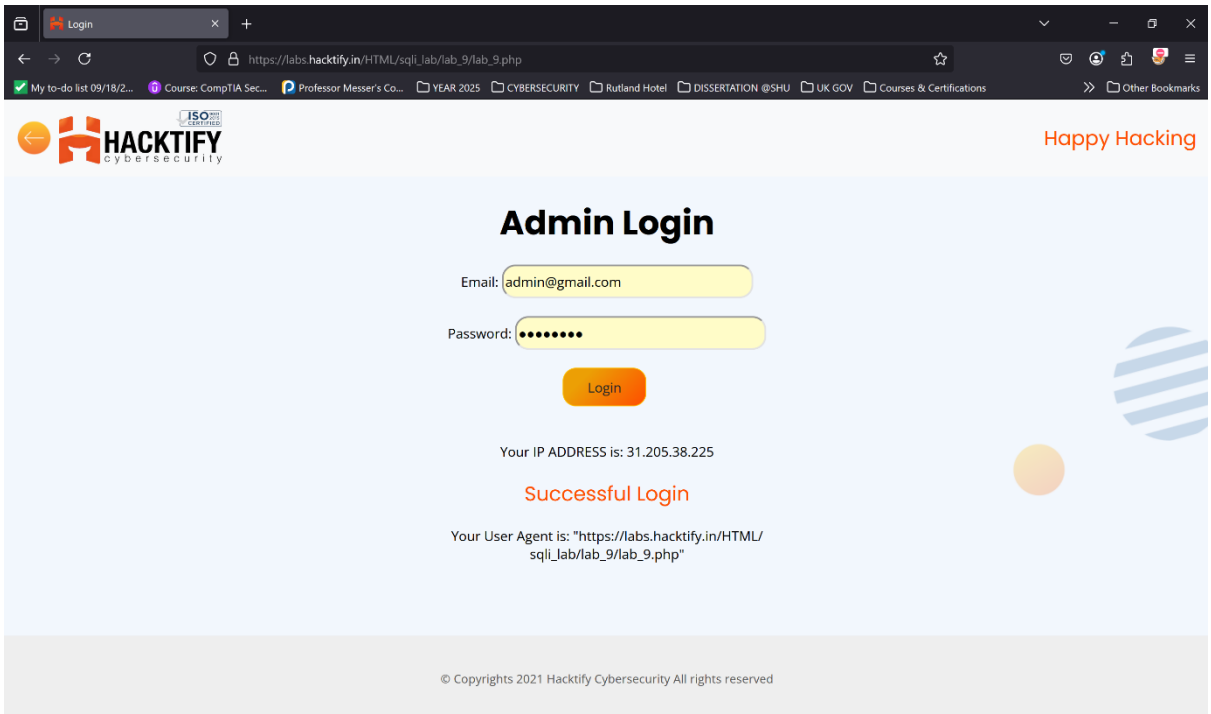
1. **Use Parameterized Queries:** Ensure that all SQL queries use prepared statements with bound parameters to prevent SQL injection.
2. **Sanitize and Validate Input:** Strictly validate and sanitize all user inputs, including HTTP headers, before processing them.
3. **Implement Web Application Firewall (WAF):** A WAF can help detect and block SQL injection attempts before they reach the application.

References

https://owasp.org/www-community/attacks/SQL_Injection
https://cwe.mitre.org/data/definitions/89.html
https://portswigger.net/web-security/sql-injection
https://nvd.nist.gov/vuln/detail/CVE-2022-1234

Proof of Concept

This section contains the proof of the above vulnerabilities as the screenshot of the vulnerability of the lab



Burp Project Intruder Repeater View Help — Burp Suite Community Edition v2025.1.1 - Temporary Project

Dashboard | Target | Proxy | Intruder | Repeater | Collaborator | Sequencer | Decoder | Comparer | Logger | Organizer | Extensions | Learn | Settings

Intercept | HTTP history | WebSockets history | Match and replace | Proxy settings

Intercept on | Forward | Drop | Request to https://labs.hacktify.in:443 [162.0.229.223] | Open browser

Time	Type	Direction	Method	URL	Status code	Length
15:10:42 20 F...	HTTP	→ Request	POST	https://labs.hacktify.in/HTML/sqli_lab/lab_9/lab_9.php		

Request — Pretty | Raw | Hex

```
POST /HTML/sqli_lab/lab_9/lab_9.php HTTP/2
Host: labs.hacktify.in
Cookie: _ga_BC1TF45GTQ=GS1.1.1739523174.1.1.1739523193.0.0.0; _ga=GA1.1.271810067.1739523175; PHPSESSID=cd3a79f5d5b28e27ac28e87668135c8b
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:135.0) Gecko/20100101 Firefox/135.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-GB,en;q=0.5
Accept-Encoding: gzip, deflate, br
Content-Type: application/x-www-form-urlencoded
Content-Length: 44
Origin: https://labs.hacktify.in
Referer: 1" OR "1"="1
Upgrade-Insecure-Requests: 1
Sec-Fetch-Dest: document
Sec-Fetch-Mode: navigate
Sec-Fetch-Site: same-origin
Sec-Fetch-User: ?1
Priority: u=0, i
Te: trailers

email=admin%40gmail.com&pwd=admin123&submit=
```

Inspector

Selection 21 (0x15)

Selected text: `Referer: 1" OR "1"="1`

Decoded from: Select — `Referer: 1" OR "1"="1`

Cancel | Apply changes

Request attributes 2
Request query parameters 0
Request body parameters 3
Request cookies 3
Request headers 22

Search — 0 highlights

Event log (1) | All issues — Memory: 144.5MB

Login — https://labs.hacktify.in/HTML/sqli_lab/lab_9/lab_9.php

My to-do list 09/18/2... | Course: CompTIA Sec... | Professor Messer's Co... | YEAR 2025 | CYBERSECURITY | Rutland Hotel | DISSERTATION @SHU | UK GOV | Courses & Certifications | Other Bookmarks

HACKTIFY cybersecurity — Happy Hacking

Admin Login

Email: admin@gmail.com

Password: ••••••••

Login

Your IP ADDRESS is: 31.205.38.225

Successful Login

Your User Agent is: "1" OR "1"="1"

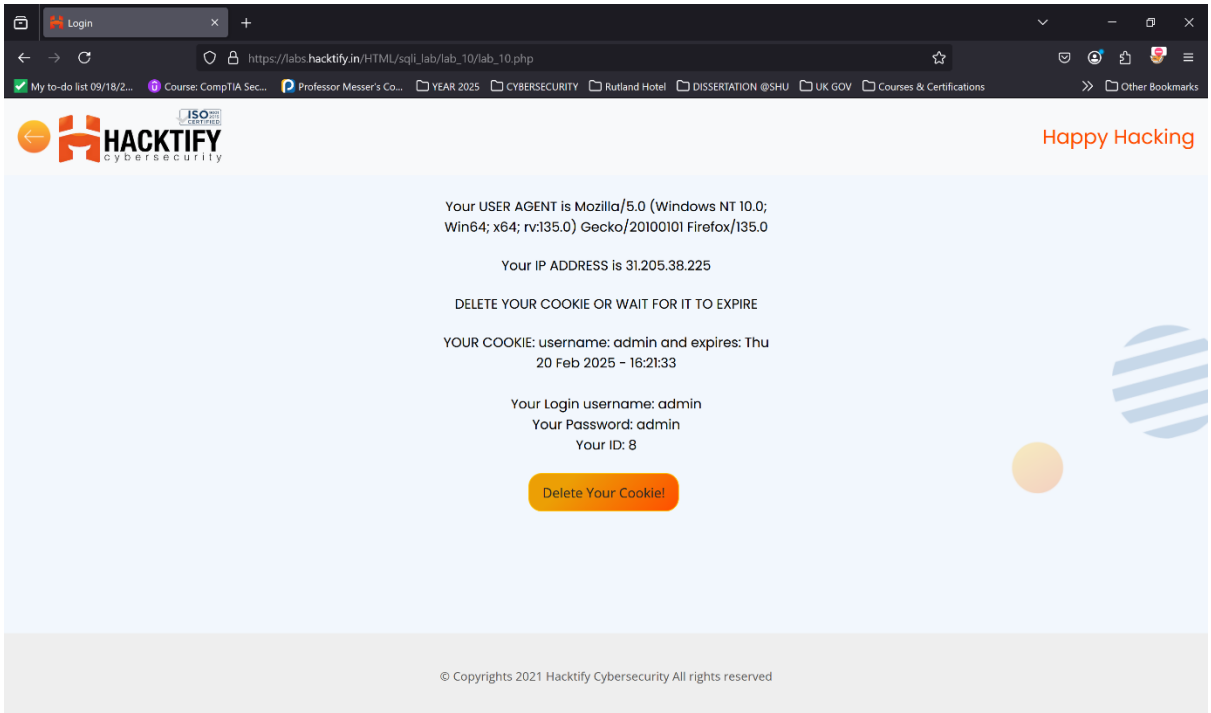
© Copyrights 2021 Hacktify Cybersecurity All rights reserved

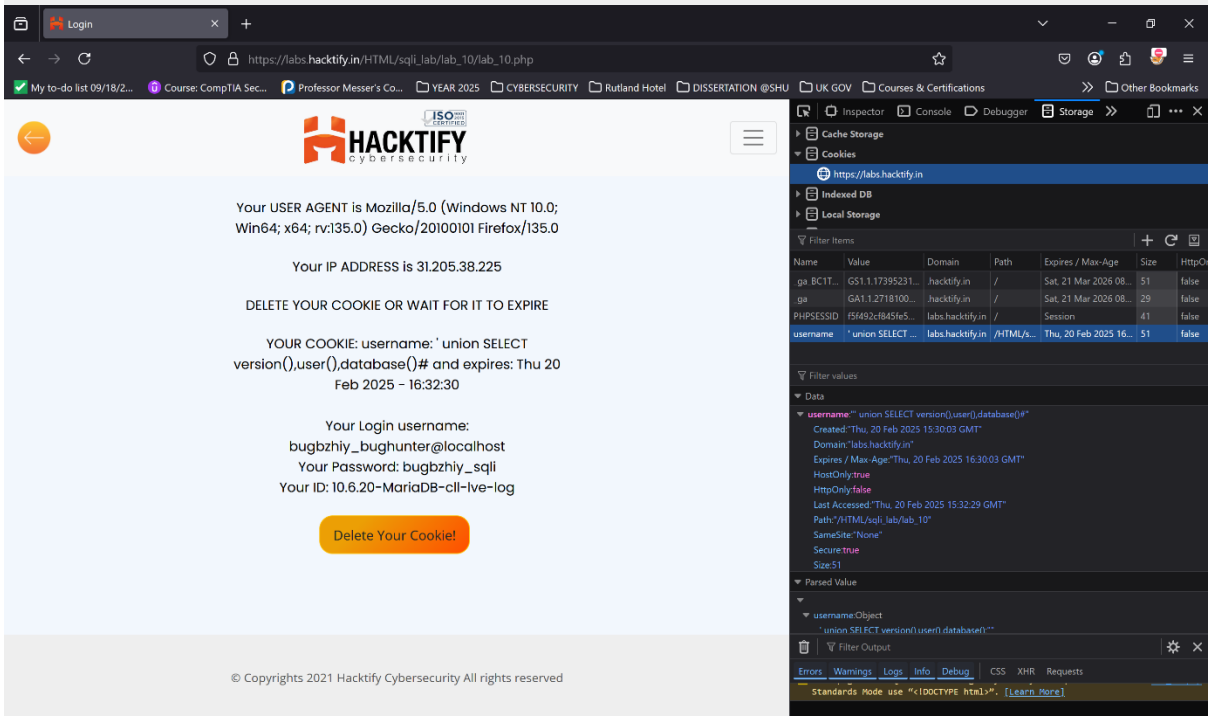
2.10. Oh Cookies!

Reference	Risk Rating
Sub-lab-10: Oh Cookies!	**High**
Tools Used	
Web Browser “Inspector tool” is used to find this vulnerability.	
Vulnerability Description	
This demonstrates a Cookie-based SQL Injection vulnerability. The application uses the value of the 'username' cookie in an SQL query without proper sanitization or escaping. By injecting a malicious SQL payload ' union SELECT version(),user(),database()# into the 'username' cookie, an attacker can execute arbitrary SQL commands. In this case, the attacker retrieves the database version, current user, and database name. The # symbol is used to comment out the rest of the original query, preventing errors.	
How It Was Discovered	
Manual Analysis	
Vulnerable URLs	
https://labs.hacktify.in/HTML/sqli_lab/lab_10/lab_10.php	
Consequences of not Fixing the Issue	
1. **Data Breach:** An attacker could potentially extract sensitive data from the database, including user credentials, personal information, or financial records. 2. **Database Manipulation:** An attacker could modify or delete data in the database, leading to data corruption or loss. 3. **Privilege Escalation:** An attacker could potentially gain higher privileges within the application or the database server. 4. **Denial of Service:** An attacker could potentially disrupt the application's availability by causing database errors or overloading the server.	
Suggested Countermeasures	
1. **Input Sanitization/Escaping:** Properly sanitize or escape all user-provided data, including cookie values, before using them in SQL queries. Use parameterized queries or prepared statements, which separate the data from the SQL code. 2. **Principle of Least Privilege:** Ensure that the database user account used by the application has only the necessary permissions to perform its tasks. Avoid using a highly privileged account. 3. **Cookie Security:** Implement HTTPOnly flag for cookies to prevent client-side scripts from accessing the 'username' cookie. Also, consider encrypting the cookie value.	
References	
https://owasp.org/www-community/attacks/SQL_Injection https://portswigger.net/web-security/sql-injection https://www.sans.org/blog/sql-injection-prevention-cheat-sheet/ https://www.acunetix.com/websitesecurity/sql-injection/	

Proof of Concept

This section contains the proof of the above vulnerabilities as the screenshot of the vulnerability of the lab


Login
https://labs.hacktify.in/HTML/sqli_lab/lab_10/lab_10.php
My to-do list 09/18/2...
Course: CompTIA Sec...
Professor Messer's Co...
YEAR 2025
CYBERSECURITY
Rutland Hotel
DISSERTATION @SHU
UK GOV
Courses & Certifications
Other Bookmarks
HACKTIFY
cybersecurity
Happy Hacking
Your USER AGENT is Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:135.0) Gecko/20100101 Firefox/135.0
Your IP ADDRESS is 31.205.38.225
DELETE YOUR COOKIE OR WAIT FOR IT TO EXPIRE
YOUR COOKIE: username: admin and expires: Thu 20 Feb 2025 - 16:21:33
Your Login username: admin
Your Password: admin
Your ID: 8
Delete Your Cookie!
© Copyrights 2021 Hacktify Cybersecurity All rights reserved


Login
https://labs.hacktify.in/HTML/sqli_lab/lab_10/lab_10.php
My to-do list 09/18/2...
Course: CompTIA Sec...
Professor Messer's Co...
YEAR 2025
CYBERSECURITY
Rutland Hotel
DISSERTATION @SHU
UK GOV
Courses & Certifications
Other Bookmarks
HACKTIFY
cybersecurity
Your USER AGENT is Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:135.0) Gecko/20100101 Firefox/135.0
Your IP ADDRESS is 31.205.38.225
DELETE YOUR COOKIE OR WAIT FOR IT TO EXPIRE
YOUR COOKIE: username: ' union SELECT version(),user(),database()# and expires: Thu 20 Feb 2025 - 16:32:30
Your Login username: bugbzhiy_bughunter@localhost
Your Password: bugbzhiy_sqli
Your ID: 10.6.20-MariaDB-cll-lve-log
Delete Your Cookie!
© Copyrights 2021 Hacktify Cybersecurity All rights reserved
Inspector
Console
Debugger
Storage
Cache Storage
Cookies
https://labs.hacktify.in
Indexed DB
Local Storage
Filter Items
Name
Value
Domain
Path
Expires / Max-Age
Size
HttpOn
_ga_BC1T...
GS1.1.1739523 1...
.hacktify.in
/
Sat, 21 Mar 2026 08...
51
false
_ga
GA1.1.2718100...
.hacktify.in
/
Sat, 21 Mar 2026 08...
29
false
PHPSESSID
f5f492cf845fe5...
labs.hacktify.in
/
Session
41
false
username
' union SELECT ...
labs.hacktify.in
/HTML/s...
Thu, 20 Feb 2025 16...
51
false
Filter values
Data
username:" union SELECT version(),user(),database()#"
Created:"Thu, 20 Feb 2025 15:30:03 GMT"
Domain:"labs.hacktify.in"
Expires / Max-Age:"Thu, 20 Feb 2025 16:30:03 GMT"
HostOnly:true
HttpOnly:false
Last Accessed:"Thu, 20 Feb 2025 15:32:29 GMT"
Path:"/HTML/sqli_lab/lab_10"
SameSite:"None"
Secure:true
Size:51
Parsed Value
username:Object
Filter Output
Errors
Warnings
Logs
Info
Debug
CSS
XHR
Requests
Standards Mode use "<!DOCTYPE html>". [Learn More]

2.11. WAF's are injected!

Reference	Risk Rating
Sub-lab-11: WAF's are injected!	**High**

Tools Used

Web Browser is used to find this vulnerability.

Vulnerability Description

The application is vulnerable to SQL Injection. The id parameter in the URL is directly incorporated into an SQL query without proper sanitization or escaping.
The injected payload ?id=1&id=0' +union+select+1,@@version,database()--+ exploits this flaw: - ': Closes the original query. - union select: Appends a new query. - 1,@@version,database(): Retrieves the database version and name. - --+: Comments out the rest of the original query.
This allows an attacker to retrieve sensitive information (in this case, database version and name) directly from the database.
The error showing "Attack Blocked Successfully by WAF" indicates a Web Application Firewall (WAF) is present but can be bypassed, possibly due to signature evasion or weak rules.

How It Was Discovered

Manual Analysis

Vulnerable URLs

https://labs.hacktify.in/HTML/sqli_lab/lab_11/lab_11.php

Consequences of not Fixing the Issue

1. **Data Breach:** Attackers can access and exfiltrate sensitive data, including user credentials, personal information, and financial data.
2. **Database Manipulation:** Attackers can modify or delete data, leading to data corruption or loss.
3. **Account Takeover:** If user credentials are compromised, attackers can gain unauthorized access to user accounts.
4. **Full System Compromise:** In severe cases, attackers can use SQL Injection to execute arbitrary code on the server, leading to a complete system compromise.

Suggested Countermeasures

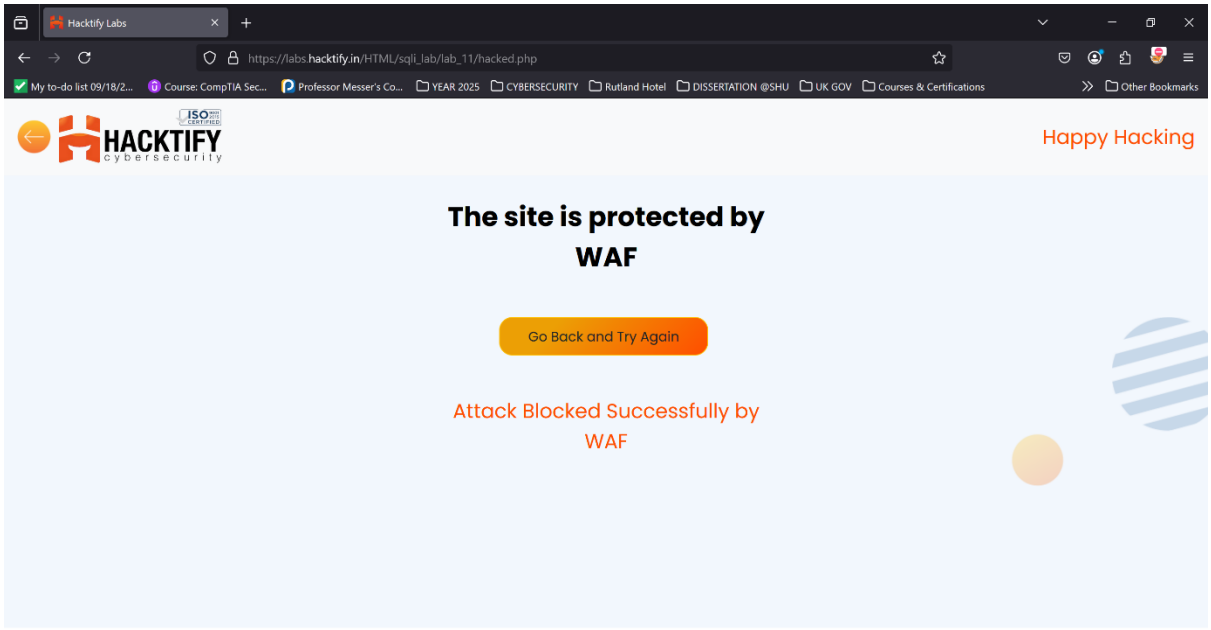
1. **Parameterized Queries/Prepared Statements:** Use parameterized queries or prepared statements to ensure that user input is treated as data and not as executable code.
2. **Input Validation and Sanitization:** Implement strict input validation to filter out malicious characters and patterns. Sanitize user input by escaping special characters.
3. **WAF Configuration:** Strengthen WAF rules to detect and block a wider range of SQL injection attempts. Regularly update WAF rules to address new attack vectors.

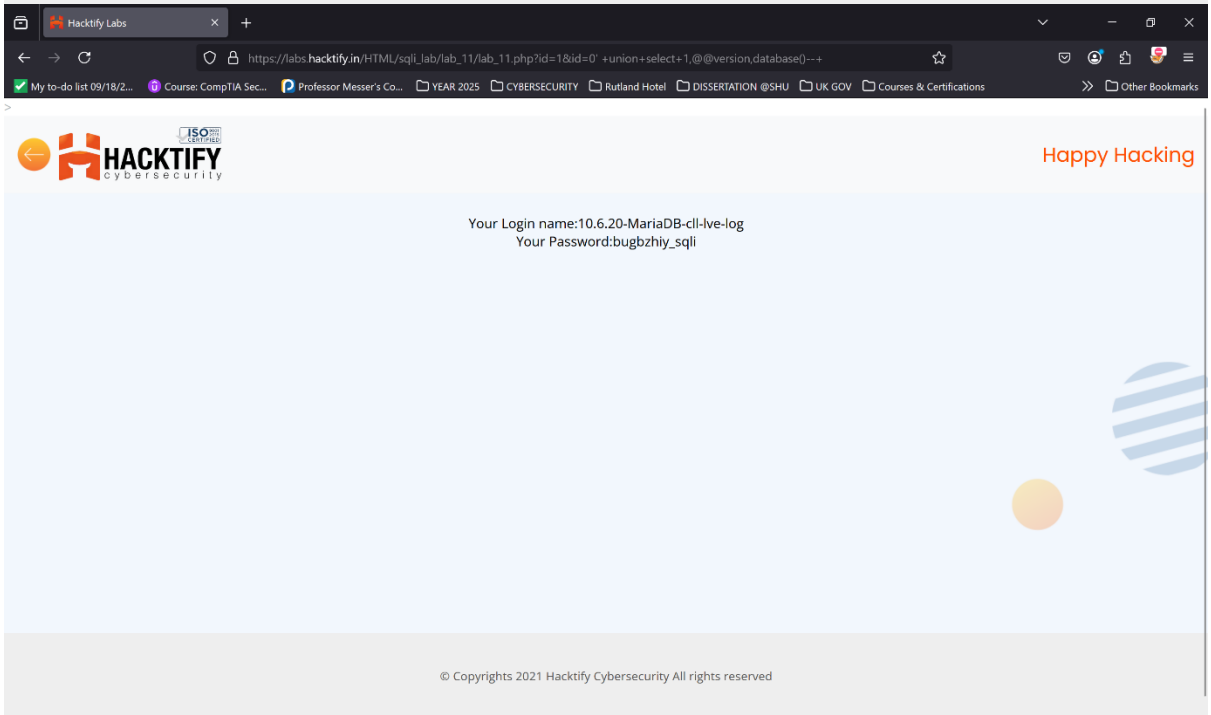
References

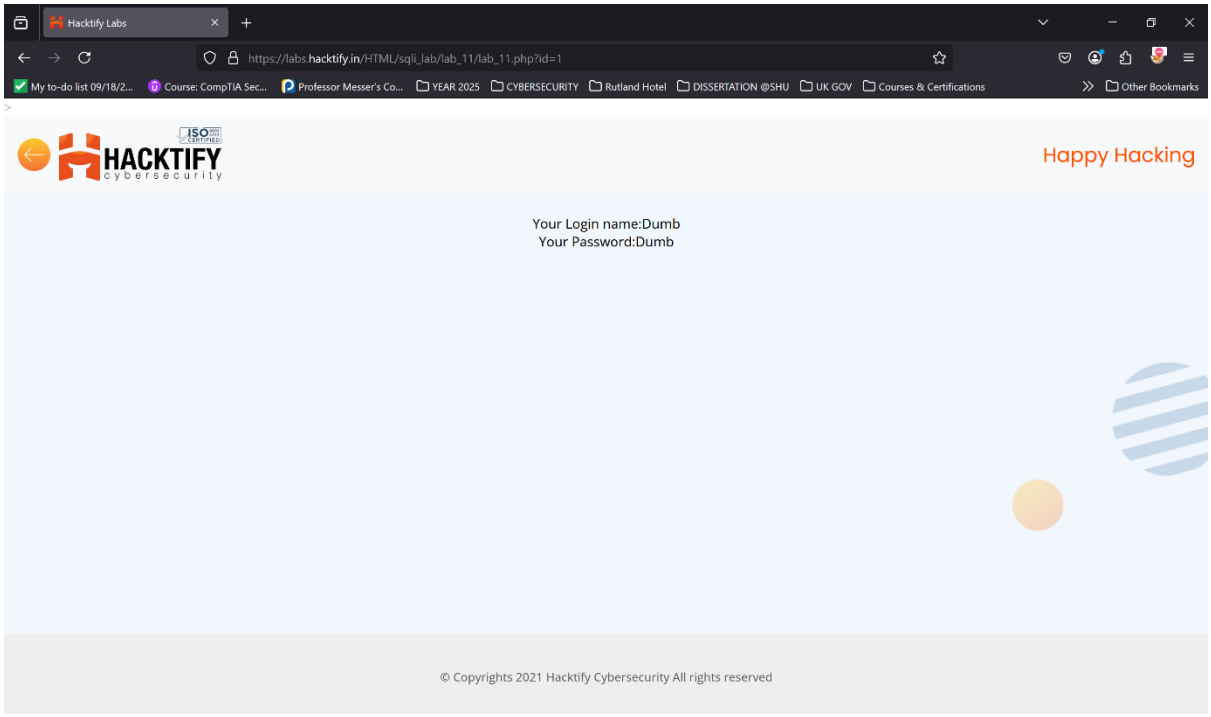
1. OWASP SQL Injection: https://owasp.org/www-community/attacks/SQL_Injection
2. SQL Injection Prevention Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html
3. PortSwigger SQL Injection: https://portswigger.net/web-security/sql-injection
4. Acunetix SQL Injection: https://www.acunetix.com/websitesecurity/sql-injection/

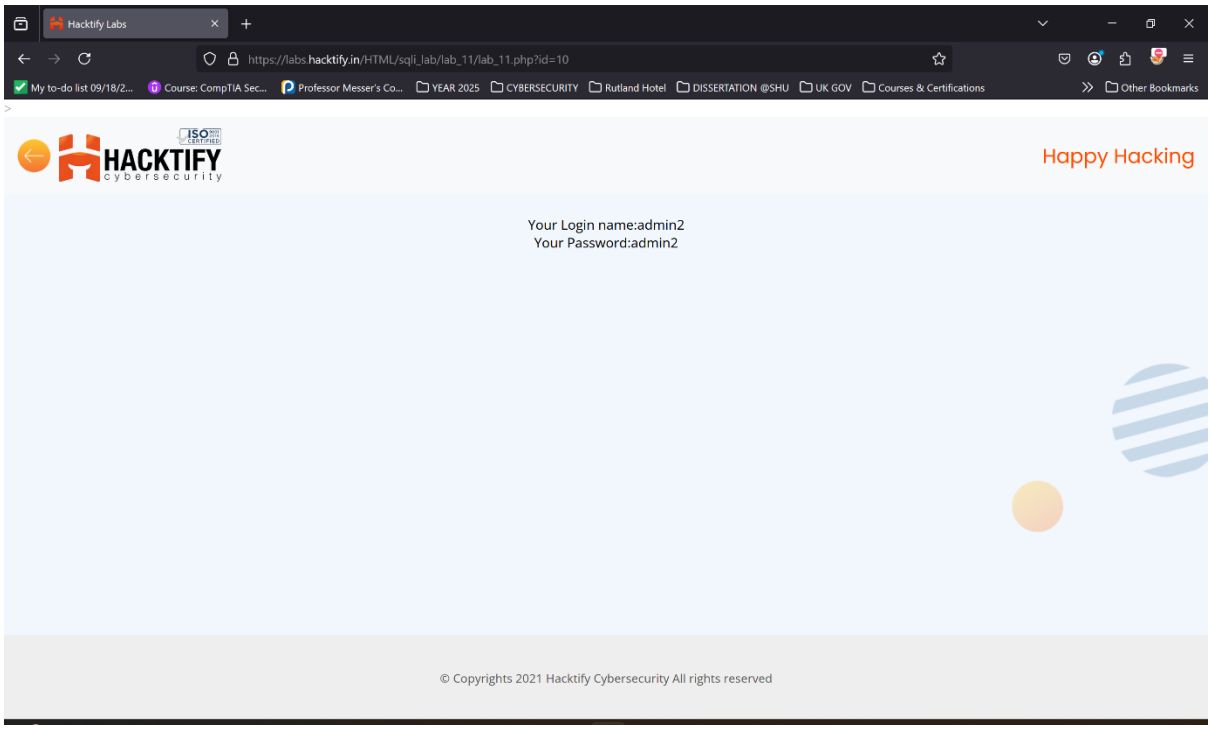
Proof of Concept

This section contains the proof of the above vulnerabilities as the screenshot of the vulnerability of the lab



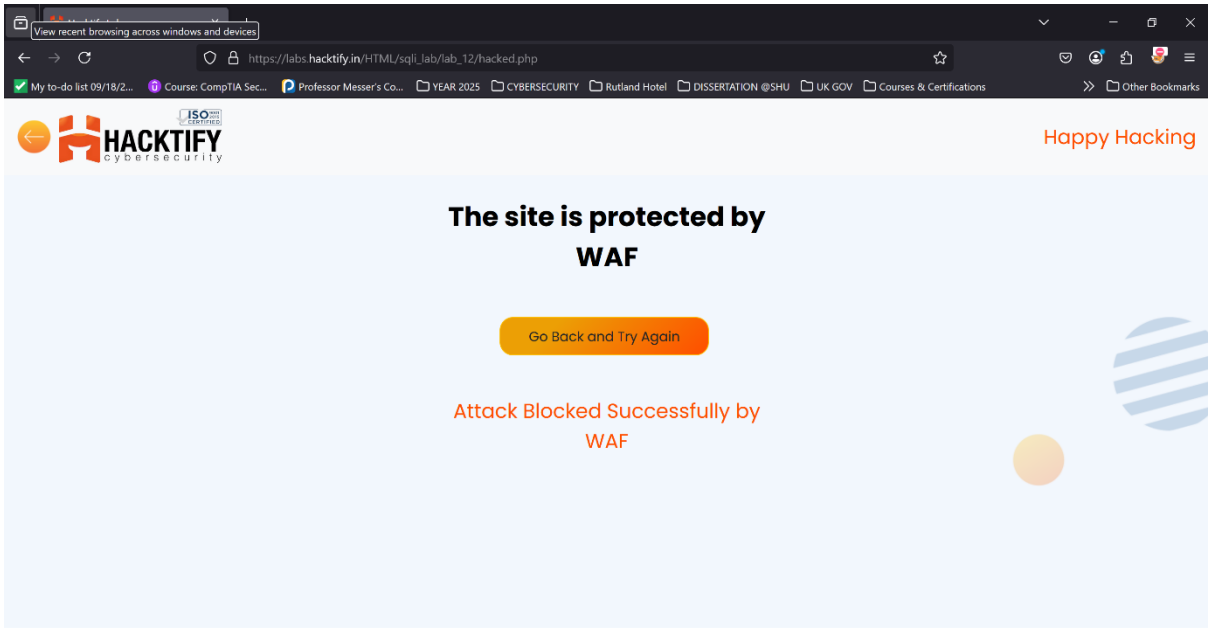
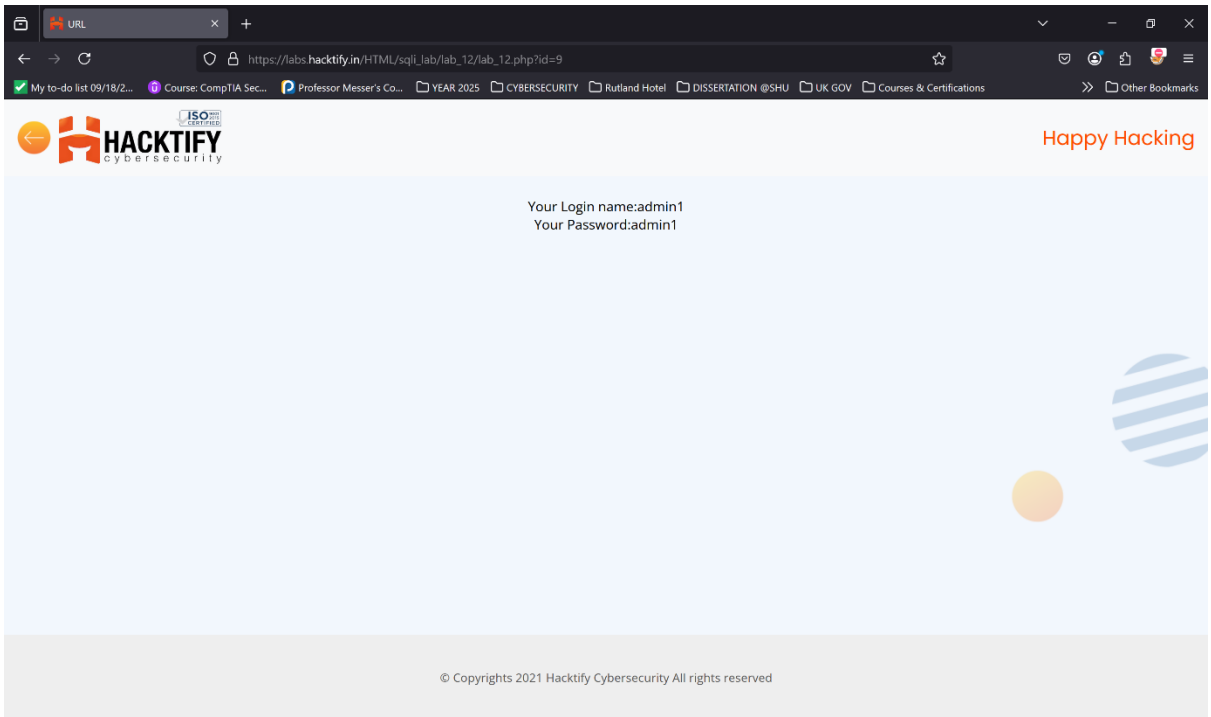



Hacktify Labs
https://labs.hacktify.in/HTML/sqli_lab/lab_11/lab_11.php?id=1
My to-do list 09/18/2...
Course: CompTIA Sec...
Professor Messer's Co...
YEAR 2025
CYBERSECURITY
Rutland Hotel
DISSERTATION @SHU
UK GOV
Courses & Certifications
Other Bookmarks
HACKTIFY
cybersecurity
Happy Hacking
Your Login name:Dumb
Your Password:Dumb
© Copyrights 2021 Hacktify Cybersecurity All rights reserved


Hacktify Labs
https://labs.hacktify.in/HTML/sqli_lab/lab_11/lab_11.php?id=10
My to-do list 09/18/2...
Course: CompTIA Sec...
Professor Messer's Co...
YEAR 2025
CYBERSECURITY
Rutland Hotel
DISSERTATION @SHU
UK GOV
Courses & Certifications
Other Bookmarks
HACKTIFY
cybersecurity
Happy Hacking
Your Login name:admin2
Your Password:admin2
© Copyrights 2021 Hacktify Cybersecurity All rights reserved

2.12. WAF's are injected Part 2!

Reference	Risk Rating
Sub-lab-11: WAF's are injected Part 2!	**Medium**
Tools Used	
Web Browser is used to find this vulnerability.	
Vulnerability Description	
This is a SQL Injection vulnerability, specifically in how the application handles the 'id' parameter in the URL. The presence of a WAF suggests an attempt to mitigate such vulnerabilities, but it appears bypassable. When the application successfully parses the '?id=9' payload without triggering the WAF, it displays sensitive information such as login credentials ("Your Login name: admin1", "Your Password: admin1").	
How It Was Discovered	
Manual Analysis	
Vulnerable URLs	
https://labs.hacktify.in/HTML/sqli_lab/lab_12/lab_12.php	
Consequences of not Fixing the Issue	
1**. Data Breach**: Attackers could extract sensitive data, including usernames, passwords, and other confidential information stored in the database. 2**. Account Takeover**: With exposed credentials, attackers can log in as legitimate users, gaining unauthorized access to accounts and potentially elevating privileges. 3. **Reputation Damage**: A successful attack can severely damage the organization's reputation, leading to a loss of customer trust and potential legal repercussions. 4**. System Compromise**: In some cases, successful SQL injection can lead to complete compromise of the underlying server, allowing attackers to execute arbitrary code and potentially use the system as a launchpad for further attacks.	
Suggested Countermeasures	
1. **Parameterized Queries/Prepared Statements**: Implement parameterized queries or prepared statements in the application code. This ensures that user-supplied input is treated as data, not as executable code, effectively preventing SQL injection. 2**. Input Validation and Sanitization**: Implement robust input validation and sanitization on the server side. Sanitize the input by encoding, or escaping special characters that could be interpreted as SQL syntax. Validate to ensure that the input matches the expected format and length. 3. **WAF Configuration Tuning**: Fine-tune the WAF rules to effectively detect and block a wider range of SQL injection attempts. This includes updating the WAF with the latest signature and regularly reviewing the WAF logs to identify and address any bypass techniques. Consider implementing rate limiting to prevent brute-force attacks.	
References	
https://owasp.org/www-community/attacks/SQL_Injection https://portswigger.net/web-security/sql-injection https://www.acunetix.com/websitesecurity/sql-injection/ https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html	

Proof of Concept

This section contains the proof of the above vulnerabilities as the screenshot of the vulnerability of the lab